



INSTITUTO POLITÉCNICO NACIONAL
ESCUELA SUPERIOR DE CÓMPUTO
ESCOM

Trabajo Terminal

2026-B009

**“Plataforma Web para la Denuncia Segura y Prevención de
Riesgos Digitales”**

Presentan

Edgar Jair Flores Espinosa

Jose Bryan Omar Jimenez Velazquez

Leilani Sharení Santos Rivera

Directores

Dr. Jessie Paulina Guzman Flores

M. en C. Ulises Velez Saldaña

INSTITUTO POLITÉCNICO NACIONAL



29 de Diciembre de 2026



**INSTITUTO POLITÉCNICO
NACIONAL
ESCUELA SUPERIOR DE CÓMPUTO
SUBDIRECCIÓN ACADÉMICA**



No. de TT: 2026-B009

Fecha de presentación de TT-II: 05 de
junio de 2026

Documento Técnico

**“Plataforma Web para la Denuncia Segura y Prevención
de Riesgos Digitales”**

Presentan:

**Edgar Jair Flores Espinosa, Jose Bryan Omar Jimenez Velazquez, Leilani
Shareni Santos Rivera**

Directores:

**Dr. Jessie Paulina Guzman Flores
M. en C. Ulises Velez Saldaña**

RESUMEN

En este trabajo terminal se desarrolló una plataforma web para la denuncia segura y prevención de riesgos digitales. La solución propuesta permite a los usuarios reportar situaciones de riesgo digital de manera segura y anónima, facilitando la identificación y respuesta ante estos casos. La plataforma integra funcionalidades de registro, seguimiento y análisis de denuncias, contribuyendo a un entorno digital más seguro para todos los usuarios.

Palabras clave: denuncia segura, prevención de riesgos digitales, plataforma web, seguridad digital.

¹josebryanomar2004@gmail.com

Advertencia

"Este documento contiene información desarrollada por la Escuela Superior de Cómputo del Instituto Politécnico Nacional, a partir de datos y documentos con derecho de propiedad y por lo tanto, su uso quedará restringido a las aplicaciones que explícitamente se convengan."

La aplicación no convenida exime a la escuela su responsabilidad técnica y da lugar a las consecuencias legales que para tal efecto se determinen.

Información adicional sobre este reporte técnico podrá obtenerse en:

La Subdirección Académica de la Escuela Superior de Cómputo del Instituto Politécnico Nacional, situada en Av. Juan de Dios Bátiz s/n
Teléfono: 57296000, extensión 52000.

Índice

1. Introducción	1
2. Planteamiento del Problema	2
2.1. Problema general	2
2.2. Problemas específicos	3
2.3. Causas	3
2.4. Alcances	4
2.5. Limitaciones	4
3. Estado del Arte	5
3.1. Criterios de evaluación	6
3.2. Plataformas gubernamentales	6
3.2.1. Sistema Integral de Denuncias Ciudadanas (SIDECA)	6
3.2.2. Denuncia Digital y sistemas estatales	7
3.2.3. Servicios de procuración digital: MP Virtual y denuncia anónima	7
3.3. Aplicaciones móviles y plataformas ciudadanas	7
3.3.1. Aplicaciones de denuncia georreferenciada	7
3.4. Plataformas internacionales	8
3.4.1. Whistlelink	8
3.4.2. EQS Integrity Line	8
3.4.3. NAVEX / WhistleB	8
3.4.4. Whistleblower Software	9
3.5. Plataformas académicas e institucionales	9
3.5.1. BullyButton	9
3.5.2. Ventanilla Escolar	9
3.5.3. Alerta IPN	9
3.5.4. UNAM: Denuncia en Línea	10
3.6. Tabla comparativa general	10
3.7. Brechas tecnológicas identificadas	10
4. Marco Teórico	11
4.1. Fundamentos de arquitectura de software	11
4.1.1. Atributos de calidad	11
4.1.2. Del monolito a los microservicios	12
4.1.3. Arquitectura en capas	13
4.1.4. Comunicación entre servicios	13
4.1.5. Persistencia de datos	14

4.2.	Tecnologías de construcción y ejecución	14
4.2.1.	Plataforma del lado del servidor	14
4.2.2.	Interfaz de usuario	14
4.2.3.	Contenedores	15
4.2.4.	Proxy inverso	15
4.2.5.	Control de versiones	15
4.3.	Seguridad aplicada al software	15
4.3.1.	Autenticación basada en tokens	15
4.3.2.	Almacenamiento seguro de contraseñas	16
4.3.3.	Control de acceso basado en roles	16
4.3.4.	Intercambio de recursos entre orígenes	16
4.3.5.	Cifrado en tránsito	17
4.4.	Problema de clasificación en el contexto de la DDP	17
4.4.1.	Taxonomía de denuncias	18
4.4.2.	Clasificación binaria y lógica de admisión	18
4.4.3.	Ciclo de vida de los datos en el aprendizaje supervisado	19
4.5.	Fundamentos de Procesamiento de Lenguaje Natural	19
4.5.1.	Álgebra lineal en espacios vectoriales de alta dimensión	20
4.5.2.	Preprocesamiento de la queja	21
4.6.	Representación vectorial del texto	22
4.6.1.	Modelos de bolsa de palabras y TF-IDF	22
4.6.2.	<i>Word embeddings</i> estáticos: spaCy (es_core_news_lg)	23
4.6.3.	<i>Sentence transformers</i> : SBERT paraphrase-multilingual-mpnet-base-v2	23
4.7.	Ingeniería de características híbridas	24
4.7.1.	Codificación de variables normativas	24
4.7.2.	Concatenación y factor de amplificación (α)	25
4.7.3.	Simulación de producción: entrenamiento con metadatos e inferencia en modo autónomo	26
4.8.	Modelos de aprendizaje automático evaluados	26
4.8.1.	Máquinas de soporte vectorial (SVM)	26
4.8.2.	Regresión logística	27
4.8.3.	Ensamble de árboles: Random Forest	27
4.8.4.	Ensamble por votación suave	28
4.9.	Estrategias de entrenamiento y validación robusta	28
4.9.1.	Validación cruzada <i>K</i> -fold estratificada	29
4.9.2.	Evaluación mediante métricas de desempeño	29
4.9.3.	Carga perezosa y serialización con joblib	31

5. Análisis y Diseño

31

5.1.	Descripción general de la solución	32
5.2.	Actores del sistema	32
5.3.	Especificación de requerimientos	33
5.3.1.	Requerimientos funcionales del quejoso	33
5.3.2.	Requerimientos funcionales del personal de recepción	35
5.3.3.	Requerimientos funcionales del administrador de TI	35
5.3.4.	Requerimientos no funcionales	36
5.3.5.	Reglas de negocio	37
5.4.	Casos de uso	39
5.5.	Ciclo de vida del expediente	41
5.6.	Diseño de la arquitectura	42
5.6.1.	Vista de componentes	42
5.6.2.	Vista de despliegue	43
5.6.3.	Diseño de la comunicación entre servicios	44
5.6.4.	Diseño de la seguridad y el control de acceso	44
5.6.5.	Decisiones de diseño y alternativas descartadas	45
5.7.	Diseño de la base de datos	46
5.7.1.	Correspondencia entre reglas de negocio y mecanismos de integridad	47
5.8.	Diseño del clasificador de competencia	48
5.9.	Diseño de la interfaz de usuario	48
6.	Implementación y evaluación del prototipo	49
6.1.	Entorno de desarrollo y herramientas	49
6.2.	Implementación del lado del servidor	50
6.2.1.	Estructura común de un microservicio	50
6.2.2.	Verificación de tokens y control de acceso	51
6.2.3.	Manejo centralizado de errores	51
6.2.4.	Documentación de la interfaz de programación	51
6.2.5.	Puntos de acceso implementados	52
6.3.	Implementación de las aplicaciones web	53
6.3.1.	Estructura común	53
6.3.2.	Interceptores y guardas	53
6.3.3.	Detección de cambios sin zonas	53
6.4.	Empaquetado en contenedores y despliegue	54
6.4.1.	Configuración externalizada	54
6.4.2.	Publicación segura y cifrado del canal	55
6.5.	Evaluación del clasificador de competencia	55
6.6.	Pruebas y verificación	56

6.7. Estado de avance por componente	56
7. Módulos de la Plataforma	57
7.1. Portal del quejoso	57
7.1.1. Área pública	57
7.1.2. Panel autenticado	58
7.2. Panel de revisión	58
7.2.1. Bandeja de entrada	59
7.2.2. Detalle y validación	59
7.2.3. Registro manual de quejas en papel	59
7.2.4. Historial y exportación	59
7.2.5. Emisión de acuerdos de conciliación	60
7.3. Consola de administración	60
7.3.1. Usuarios y roles	60
7.3.2. Catálogo de unidades académicas	60
7.3.3. Plantillas de documentos oficiales	60
7.3.4. Seguridad y respaldos	61
7.3.5. Administración de preguntas frecuentes	61
7.4. Relación entre módulos y casos de uso	61

Índice de figuras

Índice de tablas

3.1. Comparativa general de plataformas de denuncia digital	10
4.1. Comparación entre arquitectura monolítica y de microservicios	12
4.2. Atributos jurídicos estructurados derivados del Artículo 4.º	24
5.1. Actores, roles e interfaces del sistema	32
5.2. Requerimientos funcionales del actor Quejoso	34
5.3. Requerimientos funcionales del actor Personal de recepción	35
5.4. Requerimientos funcionales del actor Administrador de TI	36
5.5. Requerimientos no funcionales de la plataforma	36
5.6. Reglas de negocio de la plataforma	38
5.7. Casos de uso del actor Quejoso	39
5.8. Casos de uso del actor Personal de recepción	40
5.9. Casos de uso del actor Administrador de TI	41
5.10. Microservicios de la plataforma y su responsabilidad	42
5.11. Reglas de enrutamiento del proxy inverso público	43
5.12. Esquema de la base de datos por tabla y servicio propietario	46
5.13. Reglas de negocio y mecanismo que las hace cumplir	47
6.1. Herramientas empleadas y su función en el proyecto	50
6.2. Puntos de acceso principales por microservicio	52
6.3. Matriz de despliegue en contenedores	54
6.4. Estado de avance por componente al cierre de la etapa	56
7.1. Correspondencia entre módulos entregados y casos de uso	61

1. Introducción

El presente documento reúne el avance del Trabajo Terminal 2026-B009, desarrollado durante el semestre 2025-2 en colaboración con la Defensoría de los Derechos Politécnicos (DDP) del Instituto Politécnico Nacional (IPN), con el objetivo de identificar áreas de oportunidad en sus procesos internos y proponer una solución tecnológica que los optimice.

El Instituto Politécnico Nacional es una institución educativa pública mexicana consolidada como una de las principales casas de estudio del país, la cual ofrece una amplia gama de programas académicos en diversas áreas del conocimiento [1].

La transformación digital ha modificado profundamente la manera en que las instituciones administran sus procesos internos, resguardan información sensible y prestan servicios a sus usuarios. En este contexto, las organizaciones educativas enfrentan retos nuevos relacionados con la atención de quejas, la protección de los derechos de su comunidad y la necesidad de contar con mecanismos tecnológicos capaces de garantizar seguridad, trazabilidad y eficiencia operativa. Dentro del IPN, la Defensoría de los Derechos Politécnicos desempeña un papel fundamental como órgano encargado de promover, proteger y defender los derechos de estudiantes, docentes, personal administrativo y demás integrantes de la comunidad politécnica [2], [3].

Actualmente, buena parte de los procedimientos de recepción, análisis y seguimiento de quejas depende de procesos manuales o parcialmente digitalizados, lo que ocasiona diversos problemas operativos [4]. Entre las limitaciones identificadas destacan la saturación administrativa, la dependencia del criterio humano en la etapa de primer contacto, la dificultad para dar seguimiento oportuno a los expedientes y la ausencia de mecanismos inteligentes que automaticen la clasificación y canalización de casos. Estas problemáticas provocan retrasos en la atención, inconsistencias en la interpretación normativa y una capacidad institucional limitada para responder al incremento constante de solicitudes.

A partir de este escenario surge la necesidad de desarrollar una plataforma web especializada en la gestión digital de quejas y denuncias dentro de la DDP-IPN. La propuesta que aquí se presenta busca diseñar e implementar una solución tecnológica moderna que optimice el flujo operativo institucional mediante la integración de herramientas de procesamiento de lenguaje natural, aprendizaje automático, arquitectura de microservicios y mecanismos avanzados de seguridad informática. El propósito no se limita a digitalizar formularios o expedientes, sino a construir un ecosistema integral capaz de asistir al personal jurídico y administrativo en la toma de decisiones sobre la admisión y clasificación de quejas.

El proyecto se fundamenta en un análisis previo del estado del arte de plataformas gubernamentales, académicas e internacionales orientadas a la gestión de denuncias digitales [5]-[10]. Dicho análisis permitió identificar que las soluciones existentes presentan limitaciones importantes en escalabilidad, automatización inteligente, protección de identidad, interoperabilidad institucional y especialización universitaria. Mientras algunas priorizan el cumplimiento normativo y la formalización administrativa, otras se enfocan en la anonimización o la gestión corporativa, y dejan vacíos relevantes en contextos educativos donde convergen necesidades jurídicas, administrativas y de protección de derechos humanos. De ahí que se identificara una oportunidad tecnológica para desarrollar una solución adaptada a las necesidades operativas de la DDP.

Uno de los principales aportes de este trabajo consiste en la incorporación de un clasificador inteligente de quejas basado en técnicas de Procesamiento de Lenguaje Natural (PLN) y aprendizaje supervisado [11]-[13]. Este componente asiste al personal de primer contacto en la determinación de competencia o incompetencia de una queja, y toma como referencia el marco normativo establecido en el Acuerdo Número Extraordinario 622 de la Gaceta Politécnica [2]. Hoy en día esa decisión depende del análisis manual que realiza el personal administrativo y ju-

rídico, lo cual genera variabilidad en los criterios de interpretación y un consumo considerable de tiempo operativo.

Además del componente de inteligencia artificial, el proyecto contempla una arquitectura moderna basada en microservicios, orientada a garantizar modularidad, escalabilidad y mantenibilidad del sistema [14], [15]. Entre los módulos principales destacan el portal del quejoso, el módulo administrativo, el expediente digital, el sistema de seguridad y control de acceso, y el clasificador inteligente. Esta arquitectura desacopla las funcionalidades críticas y facilita futuras ampliaciones sin afectar la operación general de la plataforma.

Como parte del trabajo realizado se mantuvo una interacción constante con la DDP mediante reuniones, levantamiento de requerimientos y validación de procesos operativos. Ello permitió modelar el flujo real de atención de quejas con diagramas BPMN, identificar reglas de negocio críticas y diseñar soluciones alineadas con las necesidades institucionales. También se elaboraron prototipos funcionales, mockups, modelos de base de datos, la implementación de backend y frontend del microservicio de atención al quejoso, así como pruebas funcionales y experimentales para validar el comportamiento del sistema.

En conjunto, este trabajo presenta una propuesta integral para modernizar la gestión de quejas y denuncias dentro de la Defensoría de los Derechos Politécnicos del IPN. La integración de inteligencia artificial, arquitecturas escalables y mecanismos de seguridad avanzada sienta las bases de una plataforma institucional capaz de optimizar procesos administrativos, reducir tiempos de respuesta y fortalecer la protección de los derechos de la comunidad politécnica. En los capítulos siguientes se desarrollan los fundamentos teóricos, metodológicos y técnicos que sustentan el diseño, la implementación y la validación de la solución propuesta.

2. Planteamiento del Problema

Este capítulo presenta el análisis de las principales problemáticas identificadas en los procesos actuales de atención y gestión de quejas dentro de la DDP. Se describen las causas que originan dichas limitaciones operativas y tecnológicas, así como la propuesta de solución planteada para optimizar la administración de expedientes, mejorar la trazabilidad de la información y fortalecer la eficiencia institucional mediante una plataforma digital integral.

2.1. Problema general

La DDP enfrenta diversos retos operativos derivados de la gestión manual y parcialmente digitalizada de las quejas que presenta la comunidad politécnica. Hoy en día, buena parte del registro, la validación, el seguimiento y la resolución de expedientes descansa en mecanismos administrativos fragmentados, lo que se traduce en retrasos en la atención, dificultades de seguimiento y una capacidad limitada de supervisión institucional [4].

A ello se suma que la ausencia de una plataforma centralizada entorpece la comunicación entre las áreas que intervienen en el procedimiento y provoca inconsistencias en el flujo operativo, duplicidad de información y complicaciones para dar seguimiento oportuno a cada expediente. Esta situación repercute de forma directa en la eficiencia institucional, en la experiencia del usuario y en la capacidad de respuesta de la DDP ante situaciones que exigen atención inmediata.

La administración de expedientes y evidencias digitales presenta además limitaciones importantes de organización, consulta y resguardo seguro de la información, lo cual eleva el riesgo de

pérdida documental y de errores administrativos, y dificulta garantizar la integridad de los datos durante todo el ciclo de vida de la queja.

Por último, la determinación preliminar de competencia y procedencia depende en gran medida de la revisión manual de las narrativas que redactan los usuarios, lo que incrementa la carga operativa del personal y puede generar inconsistencias en la clasificación inicial de los casos [2]. En consecuencia, la DDP encuentra dificultades para reducir sus tiempos de respuesta, abatir el rezago administrativo y sostener un flujo de atención eficiente y transparente.

2.2. Problemas específicos

Del análisis del flujo operativo institucional y de los procesos actuales de atención de quejas se desprenden los siguientes problemas específicos:

- **Ausencia de una plataforma centralizada:** no existe un punto único para la gestión integral de expedientes, evidencias y seguimiento de casos.
- **Dependencia de procesos manuales:** el registro, la validación y la canalización de las quejas se realizan sin apoyo de herramientas automatizadas.
- **Determinación de competencia lenta e inconsistente:** resulta difícil establecer con rapidez y bajo un criterio uniforme la competencia institucional de cada caso.
- **Trazabilidad escasa:** el ciclo de vida de las quejas deja poco rastro documental y el estado actual de los expedientes tiene visibilidad limitada.
- **Retrasos administrativos:** la revisión manual de documentos y evidencias digitales alarga los tiempos de atención.
- **Comunicación deficiente entre áreas:** las distintas áreas que participan en el proceso de atención carecen de canales integrados de coordinación.
- **Riesgos en el manejo de información sensible:** las evidencias digitales se administran sin mecanismos robustos de control de acceso y auditoría.
- **Falta de validación automatizada:** no existen mecanismos que detecten información incompleta, inconsistente o capturada de forma incorrecta por los usuarios.
- **Historial poco estructurado:** resulta complicado mantener el registro ordenado de actuaciones, acuerdos, notificaciones y resoluciones de cada expediente.
- **Limitaciones para la explotación de datos:** la infraestructura actual no permite generar estadísticas, indicadores ni reportes institucionales.

2.3. Causas

Las problemáticas identificadas se originan en factores tecnológicos, operativos y organizacionales que inciden de manera directa en el desempeño de los procesos internos de la DDP.

En primer lugar, una parte importante del procedimiento administrativo recae en actividades manuales del personal institucional: validación de información, clasificación de quejas, revisión documental y seguimiento de expedientes. Esa dependencia eleva la probabilidad de errores humanos, inconsistencias operativas y retrasos en la atención.

En segundo lugar, la inexistencia de una arquitectura tecnológica centralizada fragmenta la información entre las distintas áreas y dificulta integrar los procesos institucionales de forma eficiente. Como resultado, los datos de una misma queja pueden encontrarse dispersos, duplicados o desactualizados, lo que afecta la continuidad operativa y la trazabilidad del expediente.

Un tercer factor es la ausencia de mecanismos automatizados para analizar y clasificar las narrativas que presentan los usuarios. La determinación preliminar de competencia descansa hoy en la interpretación humana de los hechos descritos en cada queja, situación que incrementa la carga operativa y abre espacio a diferencias de criterio entre operadores.

A lo anterior se añade la falta de herramientas especializadas para el control de accesos, la administración de evidencias digitales y el monitoreo de actividades, lo que limita la capacidad institucional de garantizar seguridad, privacidad e integridad documental.

Por último, el crecimiento del volumen de información y la complejidad de los procesos administrativos evidencian las limitaciones de la infraestructura tecnológica tradicional, que dificulta la escalabilidad, la mantenibilidad y la modernización de los servicios digitales institucionales.

2.4. Alcances

El proyecto delimita su alcance a la Defensoría de los Derechos Politécnicos del Instituto Politécnico Nacional y abarca únicamente los procesos internos de gestión de quejas de dicha instancia. La plataforma será accesible desde navegador web en equipos de escritorio y portátiles; no se contempla el desarrollo de una aplicación móvil nativa.

El sistema cubre el ciclo operativo completo de una queja, desde su recepción hasta su resolución, e incluye módulos diferenciados para cada rol institucional: Quejoso, Recepcionista, Área de Primer Contacto, Defensor, Subdefensor y Titular. Se integra además un módulo de clasificación automática asistida, basado en Procesamiento de Lenguaje Natural, cuya función es sugerir la competencia preliminar de cada queja [13]. En la etapa que documenta este trabajo dicho módulo opera sobre un corpus sintético construido por el equipo de desarrollo, por lo que sus resultados tienen carácter preliminar y quedan sujetos a la recalibración señalada en la Sección 2.5. El quejoso podrá consultar el estado de su caso en cualquier momento mediante un folio digital único, generado de forma automática al momento del registro.

Quedan excluidos de forma explícita la integración con sistemas externos al IPN, la gestión de procesos distintos al flujo de quejas de la DDP y la emisión automatizada de resoluciones o dictámenes legales.

2.5. Limitaciones

La ejecución del proyecto está condicionada por los siguientes factores:

- **Tiempo:** al tratarse de un proyecto académico con entregas parciales, no todos los módulos institucionales se encuentran operativos en esta etapa. Los componentes desarrollados hasta el momento son el microservicio de gestión de quejas y el módulo clasificador de competencia; los restantes quedan para una entrega posterior.
- **Datos:** el entrenamiento y la validación del clasificador se realizaron con un corpus sintético de quejas, construido por el equipo de desarrollo. A la fecha de elaboración de este documento, la DDP aún no ha entregado los casos históricos reales y anonimizados solicitados, cuya gestión continúa en trámite. Dado que la calidad y el volumen de los datos

de entrenamiento inciden directamente en la precisión del modelo, las métricas reportadas en este trabajo deben considerarse preliminares y quedan sujetas a recalibración una vez que se disponga de los expedientes reales.

- **Adopción institucional:** la efectividad del sistema depende de la disposición del personal de la DDP para incorporar la plataforma a sus procesos cotidianos y validar las clasificaciones que sugiere el modelo de inteligencia artificial.
- **Restricción ética y normativa:** el sistema no sustituye en ningún caso el criterio jurídico humano. Las resoluciones finales son responsabilidad exclusiva del personal de la Defensoría, conforme a las reglas de negocio establecidas y a la normativa aplicable en materia de protección de datos personales.

3. Estado del Arte

La transformación digital ha impulsado el desarrollo de plataformas orientadas a la gestión electrónica de denuncias, quejas y mecanismos de protección de derechos. Organizaciones gubernamentales, corporativas y académicas han incorporado sistemas capaces de automatizar la recepción de reportes, el seguimiento de expedientes y la administración de evidencia electrónica.

Esa evolución, sin embargo, no ha sido homogénea. Algunos sistemas privilegian la validez jurídica y el cumplimiento normativo, mientras que otros se concentran en la anonimización de datos, la movilidad, la trazabilidad o la incorporación de mecanismos de seguridad informática. El resultado es que buena parte de las soluciones disponibles resulta insuficiente para contextos institucionales complejos, en particular para entornos universitarios donde convergen aspectos administrativos, jurídicos, académicos y de protección de derechos humanos.

En el caso de la DDP-IPN, una plataforma especializada debe atender exigencias que rebasan las de un sistema convencional de recepción de denuncias. Los aspectos críticos identificados son los siguientes:

- Protección e integridad de la información sensible.
- Automatización de procesos administrativos.
- Escalabilidad de la infraestructura tecnológica.
- Trazabilidad de expedientes digitales.
- Protección de identidad y anonimización del denunciante.
- Integración institucional entre distintas áreas administrativas.
- Adaptabilidad a procesos universitarios complejos.

Con ese marco, este capítulo analiza diversas plataformas nacionales e internacionales de denuncia digital y gestión institucional, con el objetivo de identificar las capacidades tecnológicas existentes, las limitaciones arquitectónicas predominantes y las brechas funcionales que justifican el desarrollo de una solución especializada para el Instituto Politécnico Nacional.

3.1. Criterios de evaluación

Para analizar las plataformas seleccionadas bajo una misma óptica, se definieron criterios de evaluación que abarcan tanto aspectos funcionales como arquitectónicos:

- **Escalabilidad arquitectónica:** capacidad del sistema para absorber crecimiento en usuarios, servicios y procesamiento sin degradar de forma significativa su rendimiento.
- **Seguridad y confidencialidad:** mecanismos con que protege la información sensible, entre ellos cifrado, autenticación y control de acceso.
- **Anonimización y protección de identidad:** nivel de protección que ofrece al denunciante para preservar su identidad y confidencialidad.
- **Trazabilidad y seguimiento:** posibilidad de monitorear el estado y el avance de los expedientes o denuncias digitales.
- **Automatización administrativa:** uso de tecnologías que reducen el trabajo manual mediante reglas de negocio, clasificación automática o asistencia inteligente.
- **Soporte multimedia:** capacidad de almacenar y gestionar evidencia digital como imágenes, documentos, audio y video.
- **Interoperabilidad institucional:** facilidad para integrarse con otros servicios, sistemas o plataformas de la propia institución.
- **Adaptación al entorno universitario:** grado de adecuación funcional y administrativa a instituciones educativas públicas.

Sobre estos criterios se construyó el análisis comparativo que se presenta a continuación.

3.2. Plataformas gubernamentales

Esta sección revisa las herramientas tecnológicas que el Estado mexicano ha implementado para recibir, gestionar y dar seguimiento a denuncias ciudadanas. A partir de soluciones federales, como el Sistema Integral de Denuncias Ciudadanas (SIDECE), y de iniciativas locales, como la plataforma Denuncia Digital de la Ciudad de México, se examinan sus capacidades funcionales, su arquitectura y los retos que aún enfrentan en materia de interoperabilidad, automatización y descentralización.

3.2.1. Sistema Integral de Denuncias Ciudadanas (SIDECE)

El SIDECE es una de las principales plataformas gubernamentales de recepción de quejas administrativas en México y centraliza las denuncias relacionadas con servidores públicos federales [5].

En el plano funcional, el sistema recibe denuncias de forma continua mediante formularios electrónicos y ofrece mecanismos básicos de seguimiento a través de folios únicos y contraseñas de acceso [16]. Admite además evidencia multimedia en formato de documento, audio y video. Su relevancia como referente se refuerza con los trabajos de mejora que la Secretaría de la Función Pública ha impulsado junto con el BID y GovLab [17].

El análisis arquitectónico, en cambio, revela varias limitaciones técnicas:

- Arquitectura predominantemente centralizada.
- Procesos administrativos parcialmente manuales.
- Seguimiento limitado para el denunciante.
- Ausencia de automatización inteligente.
- Escasa modularidad de servicios.

3.2.2. Denuncia Digital y sistemas estatales

Diversas entidades federativas han incorporado plataformas digitales para recibir denuncias y querellas. El caso más representativo es el sistema Denuncia Digital de la Ciudad de México [6], que permite iniciar procedimientos por delitos no violentos mediante autenticación electrónica con la Llave CDMX o con firma electrónica gubernamental.

La comparación entre entidades revela una fragmentación tecnológica considerable [18]: algunas plataformas ya admiten videodenuncias y ratificación remota, mientras otras siguen operando con modelos híbridos o trámites presenciales. Aunque representan un avance real en digitalización administrativa, estas soluciones se quedan cortas en interoperabilidad, automatización y protección avanzada de identidad.

3.2.3. Servicios de procuración digital: MP Virtual y denuncia anónima

La Fiscalía General de Justicia de la Ciudad de México ha sumado servicios digitales complementarios, entre ellos el MP Virtual y el sistema de denuncia anónima [19].

El módulo MP Virtual formaliza a distancia determinados procedimientos jurídicos, lo que facilita la validación documental y reduce la necesidad de comparecencia presencial [20]. El sistema de denuncia anónima, por su parte, implementa mecanismos básicos de protección de identidad para reportes de delitos de alto impacto.

Ambos servicios, no obstante, siguen apoyados en infraestructuras centralizadas y procesos administrativos tradicionales, y carecen de componentes de inteligencia artificial, clasificación automática y analítica institucional.

3.3. Aplicaciones móviles y plataformas ciudadanas

Junto a las plataformas web institucionales ha surgido una generación de aplicaciones móviles y canales ciudadanos que privilegian la accesibilidad y la participación colaborativa por encima de la formalidad administrativa.

3.3.1. Aplicaciones de denuncia georreferenciada

La evolución de los servicios digitales ha impulsado aplicaciones móviles orientadas a la denuncia ciudadana y la participación colaborativa. Uno de los referentes más conocidos es la aplicación Incorruptible, diseñada para registrar actos de corrupción mediante geolocalización [21]. Sus funcionalidades principales son:

- Registro georreferenciado de incidentes.

- Publicación colaborativa de reportes.
- Integración de evidencia multimedia.
- Cuantificación económica de los daños reportados.

En una línea complementaria, el Sistema de Atención Mexiquense (SAM) integra canales web, móviles y telefónicos para canalizar denuncias hacia distintas dependencias administrativas [22].

Su accesibilidad y facilidad de uso son evidentes, pero este tipo de aplicaciones queda limitado en trazabilidad jurídica, validación institucional y protección de datos sensibles.

3.4. Plataformas internacionales

El mercado internacional de plataformas de denuncia y cumplimiento normativo ofrece soluciones maduras, dirigidas sobre todo a sectores corporativos y regulados. Destacan por sus mecanismos de seguridad, anonimización y administración de casos, aunque suelen tropezar con costos de implementación elevados, dependencia de proveedores externos y dificultades para adaptarse a marcos institucionales específicos.

3.4.1. Whistlelink

Whistlelink gestiona denuncias corporativas de forma anónima y destaca por su soporte multilingüe y por sus canales bidireccionales de comunicación anónima [7]. Su mayor virtud es reducir las barreras de acceso para denunciante de distintos países; en contraparte, ofrece poca personalización institucional y se adapta con dificultad a procesos administrativos complejos.

3.4.2. EQS Integrity Line

EQS Integrity Line es una de las soluciones corporativas más robustas del mercado europeo, diseñada para cumplir con la normativa internacional de protección al denunciante [8]. Incorpora herramientas avanzadas de administración de casos, monitoreo organizacional y generación de reportes. Su complejidad operativa y la carga administrativa que exige, sin embargo, complican su implementación en instituciones educativas públicas.

3.4.3. NAVEX / WhistleB

NAVEX y su solución WhistleB se apoyan en infraestructura en la nube para garantizar alta disponibilidad y escalabilidad [9]. Entre sus fortalezas figuran:

- Integración con plataformas GRC.
- Infraestructura escalable.
- Gestión centralizada de cumplimiento.
- Administración avanzada de casos.

La dependencia de proveedores externos, en cambio, plantea restricciones de soberanía de datos que chocan con las políticas de las instituciones gubernamentales.

3.4.4. Whistleblower Software

Whistleblower Software sobresale por sus mecanismos de cifrado y por adoptar un modelo de arquitectura *Zero-Knowledge* [23], enfoque que refuerza de manera notable la confidencialidad de la información y la protección del denunciante. Su orientación al entorno corporativo, no obstante, limita su encaje directo en estructuras universitarias públicas.

3.5. Plataformas académicas e institucionales

Esta sección examina las herramientas e iniciativas digitales que instituciones educativas y organismos públicos han desarrollado para atender incidencias en el ámbito académico. Mediante el análisis de BullyButton, Ventanilla Escolar, Alerta IPN y el sistema de Denuncia en Línea de la UNAM se evalúan sus alcances operativos y protocolos de respuesta, y se identifican las brechas tecnológicas comunes a estos entornos: sobre todo la gestión jurídica de expedientes, la automatización del triaje y la falta de mecanismos avanzados para proteger la identidad de los usuarios.

3.5.1. BullyButton

BullyButton se enfoca en la detección y el reporte de casos de acoso escolar [24]. Su diseño privilegia la simplicidad operativa y la facilidad de uso para estudiantes, pero carece de mecanismos formales de trazabilidad jurídica y de administración avanzada de expedientes.

3.5.2. Ventanilla Escolar

Ventanilla Escolar opera como canal de orientación y comunicación entre las instituciones educativas y las autoridades federales [25]. Centraliza información relevante para la comunidad educativa, aunque no cubre el seguimiento de denuncias ni la administración formal de expedientes.

3.5.3. Alerta IPN

Alerta IPN es uno de los principales mecanismos institucionales de reporte dentro del Instituto Politécnico Nacional [26] y se orienta sobre todo a emergencias y situaciones de riesgo dentro de las instalaciones. Sus limitaciones se concentran en:

- Gestión jurídica de expedientes.
- Clasificación administrativa automatizada.
- Seguimiento integral de casos.
- Protección avanzada de identidad.
- Automatización inteligente de procesos.

3.5.4. UNAM: Denuncia en Línea

La plataforma de Denuncia en Línea de la UNAM es uno de los referentes más relevantes de gestión universitaria de denuncias en México [10]. Su mayor fortaleza son los protocolos institucionales consolidados y los procedimientos administrativos claramente definidos. El análisis tecnológico, en cambio, evidencia limitaciones en:

- Escalabilidad arquitectónica.
- Integración de inteligencia artificial.
- Automatización del triaje.
- Modularidad de servicios.
- Gestión inteligente de expedientes.

3.6. Tabla comparativa general

Con base en los criterios definidos al inicio del capítulo, se comparó el conjunto de plataformas analizadas para contrastar sus capacidades funcionales, sus limitaciones arquitectónicas y su nivel de adaptación a entornos institucionales complejos. La Tabla 3.1 resume las características identificadas durante la investigación.

Tabla 3.1: Comparativa general de plataformas de denuncia digital

Plataforma	Seguridad	Anonimato	Seguimiento	Multi-media	Automatización	Escala-bilidad	Adaptación universitaria
SIDEC	Media	Baja	Media	Sí	Baja	Media	No
Denuncia Digital CDMX	Media	Baja	Media	Sí	Baja	Media	No
MP Virtual	Media	Media	Media	Sí	Baja	Media	No
Incorruptible	Baja	Media	Baja	Sí	Baja	Alta	No
SAM	Media	Baja	Media	Sí	Baja	Media	No
Whistlelink	Alta	Alta	Alta	Sí	Media	Alta	Parcial
EQS Integrity Line	Alta	Alta	Alta	Sí	Alta	Alta	Parcial
NAVEX / WhistleB	Alta	Alta	Alta	Sí	Alta	Alta	Parcial
Whistleblower Software	Muy alta	Muy alta	Alta	Sí	Alta	Alta	Parcial
BullyButton	Media	Media	Baja	Limitado	Baja	Media	Sí
Ventanilla Escolar	Media	Baja	Baja	No	Baja	Media	Sí
Alerta IPN	Media	Baja	Media	Sí	Baja	Media	Sí
UNAM Denuncia en Línea	Media	Media	Alta	Sí	Media	Media	Sí

3.7. Brechas tecnológicas identificadas

El análisis comparativo muestra que, si bien existen múltiples plataformas dedicadas a la gestión digital de denuncias y quejas, ninguna reúne por completo los requerimientos tecnológicos,

administrativos y de protección institucional que exige un entorno universitario público como el Instituto Politécnico Nacional. Las brechas principales son:

- Dependencia de arquitecturas centralizadas tradicionales.
- Escasa automatización inteligente de los procesos administrativos.
- Mecanismos limitados de anonimización y protección de identidad.
- Baja interoperabilidad entre servicios institucionales.
- Deficiencias en la trazabilidad integral de los expedientes digitales.
- Escasa adaptación a estructuras universitarias complejas.
- Ausencia de arquitecturas híbridas escalables pensadas para instituciones académicas públicas.

Estas limitaciones justifican el desarrollo de una plataforma especializada que combine seguridad informática avanzada, modularidad arquitectónica, automatización administrativa y adaptación específica al contexto institucional de la DDP-IPN.

4. Marco Teórico

Este capítulo reúne los fundamentos conceptuales sobre los que descansa la solución propuesta y se organiza en dos bloques. El primero, de la Sección 4.1 a la Sección 4.3, desarrolla los fundamentos de la plataforma entendida como sistema de software: los estilos arquitectónicos que justifican dividirla en servicios independientes, las tecnologías con que se construye y se ejecuta, y los mecanismos que protegen la información sensible que administra. El segundo bloque, de la Sección 4.4 en adelante, desarrolla los fundamentos del clasificador de competencia: el marco legal que define el problema, los fundamentos del Procesamiento de Lenguaje Natural que permiten interpretar la narrativa de las quejas, las técnicas de representación vectorial que traducen ese lenguaje en señales numéricas y los modelos de aprendizaje automático que aprenden a distinguir entre quejas competentes e incompetentes. El orden no es casual: el clasificador no es un sistema aislado, sino un módulo que vive dentro de la arquitectura descrita en el primer bloque.

4.1. Fundamentos de arquitectura de software

La arquitectura de software es el conjunto de decisiones estructurales de alto nivel que definen cómo se organiza un sistema: qué componentes lo integran, qué responsabilidad tiene cada uno y cómo se comunican entre sí [27]. Esas decisiones son costosas de revertir una vez que el sistema está en operación, de modo que conviene tomarlas de forma explícita y justificada desde el diseño, y no dejarlas al azar de la implementación.

4.1.1. Atributos de calidad

Una arquitectura no se evalúa únicamente por las funciones que permite construir, sino por los atributos de calidad que habilita o dificulta [28]. Los relevantes para una plataforma institucional de gestión de quejas son los siguientes:

- **Escalabilidad:** capacidad de atender un volumen creciente de usuarios y peticiones agregando recursos, sin rediseñar el sistema [29]. Se distingue entre escalamiento vertical —dar más capacidad a la misma máquina— y horizontal —agregar más instancias que reparten la carga— [30].
- **Mantenibilidad:** facilidad con que el sistema puede modificarse para corregir defectos o incorporar funciones nuevas. Depende directamente de qué tan acopladas estén sus partes.
- **Disponibilidad:** proporción del tiempo en que el sistema responde correctamente. En un sistema distribuido, el fallo de un componente no debe propagarse al resto [31].
- **Seguridad y confidencialidad:** capacidad de proteger la información frente a accesos no autorizados, atributo crítico cuando los datos administrados son denuncias que identifican a personas.
- **Trazabilidad:** posibilidad de reconstruir qué ocurrió con cada expediente y cuándo, requisito tanto operativo como jurídico en el contexto de la Defensoría.

4.1.2. Del monolito a los microservicios

En una arquitectura **monolítica** toda la funcionalidad se empaqueta y despliega como una única unidad ejecutable [32]. Este enfoque es simple de desarrollar y desplegar en etapas tempranas, pero conforme el sistema crece presenta problemas conocidos: cualquier cambio obliga a reconstruir y desplegar la aplicación completa, el escalamiento debe hacerse en bloque aunque solo un componente esté saturado, y el acoplamiento interno dificulta que varias personas trabajen en paralelo sin interferir entre sí.

Una arquitectura de **microservicios** descompone la aplicación en un conjunto de servicios pequeños e independientes, cada uno con una responsabilidad acotada, su propio ciclo de despliegue y una interfaz bien definida para comunicarse con los demás [14]. Cada servicio puede desarrollarse, desplegarse y escalarse por separado, lo que reduce el impacto de un cambio y permite dimensionar los recursos según la carga real de cada componente. La Tabla 4.1 contrasta ambos estilos.

Tabla 4.1: Comparación entre arquitectura monolítica y de microservicios

Criterio	Monolito	Microservicios
Unidad de despliegue	La aplicación completa.	Cada servicio por separado.
Escalamiento	En bloque, aunque solo un módulo lo requiera.	Selectivo, por servicio.
Acoplamiento	Alto: los módulos comparten proceso y memoria.	Bajo: se comunican por interfaces explícitas.
Impacto de un fallo	Puede detener toda la aplicación.	Acotado al servicio afectado si el diseño lo aísla.
Complejidad operativa	Baja: un solo artefacto que administrar.	Alta: más artefactos, redes y puntos de fallo.
Trabajo en equipo	Los cambios compiten sobre el mismo código.	Equipos independientes por servicio.

La elección entre ambos estilos no es absoluta. Estudios comparativos que despliegan una misma aplicación bajo los dos patrones muestran que los microservicios mejoran el aprovechamiento de recursos y la escalabilidad, pero a costa de una complejidad operativa considerablemente mayor [15]. Un sistema distribuido introduce problemas que un monolito no tiene —latencia de red, fallos parciales, consistencia entre nodos— y que deben resolverse de forma explícita [33]. Por esa razón existen posiciones intermedias, como mantener servicios independientes que comparten una única base de datos: una decisión que sacrifica autonomía de datos a cambio de simplicidad operativa, razonable a escala institucional pequeña y que conviene documentar como tal.

4.1.3. Arquitectura en capas

De forma transversal al estilo anterior, la organización interna de cada servicio suele seguir un esquema en capas, en el que la presentación, la lógica de negocio y el acceso a datos se separan en niveles con responsabilidades distintas [34]. La capa de presentación expone la interfaz —en un servicio web, los puntos de acceso HTTP—; la capa de negocio concentra las reglas del dominio; y la capa de datos se ocupa de la persistencia. La ventaja principal es que un cambio en una capa no obliga a modificar las demás mientras se respete el contrato entre ellas [35].

Esta separación resulta especialmente valiosa cuando las reglas del dominio son normativas, como en el caso de una defensoría: al concentrarlas en una sola capa, un cambio en el reglamento se traduce en modificaciones localizadas y no dispersas por toda la aplicación [36]. Sobre esa base pueden además incorporarse políticas transversales —calidad de servicio, monitoreo, control de acceso— sin reescribir la lógica de negocio [37].

4.1.4. Comunicación entre servicios

Una vez dividido el sistema, sus partes deben comunicarse. Existen dos familias de mecanismos.

La **comunicación síncrona** sigue el modelo petición-respuesta: un servicio llama a otro y espera el resultado antes de continuar. El estilo arquitectónico dominante para ello es REST (*Representational State Transfer*), formulado por Fielding, que organiza la interfaz en torno a recursos identificados por URI y manipulados mediante los verbos del protocolo HTTP, con comunicación sin estado entre peticiones [38]. Su ventaja es la simplicidad conceptual y la interoperabilidad; su desventaja, que acopla temporalmente a los servicios, pues si el servicio invocado no responde, el invocador queda bloqueado o debe manejar el fallo.

La **comunicación asíncrona** desacopla al emisor del receptor mediante colas o flujos de eventos: el servicio origen publica un evento y continúa, sin esperar a que alguien lo consuma [39]. Ofrece mayor tolerancia a fallos y mejor absorción de picos de carga, al precio de una complejidad notablemente mayor y de consistencia eventual en los datos.

Cuando el número de servicios crece, la comunicación directa entre ellos se vuelve difícil de administrar y aparecen componentes de apoyo —registro y descubrimiento de servicios, configuración centralizada, balanceo de carga— que resuelven de forma estándar esa infraestructura transversal [40].

4.1.5. Persistencia de datos

La mayoría de los sistemas de información institucionales descansa sobre bases de datos relacionales, que organizan la información en tablas vinculadas por claves y garantizan las propiedades ACID: atomicidad, consistencia, aislamiento y durabilidad [41]. Esas garantías son especialmente pertinentes cuando cada registro corresponde a un expediente con efectos jurídicos, donde una escritura parcial o una lectura inconsistente no son aceptables.

PostgreSQL es un gestor relacional de código abierto ampliamente utilizado en producción, con soporte para tipos de datos binarios, transacciones concurrentes, índices avanzados y mecanismos de respaldo y restauración [42]. El diseño de la capa de datos debe considerar, además, cómo evolucionará el esquema con el tiempo y cómo se mantiene la integridad cuando varios componentes acceden al mismo almacén, problema central en el diseño de sistemas intensivos en datos [43].

4.2. Tecnologías de construcción y ejecución

Definidos los principios arquitectónicos, esta sección describe las tecnologías concretas que los materializan, agrupadas por capa.

4.2.1. Plataforma del lado del servidor

Java es un lenguaje de programación orientado a objetos, compilado a un código intermedio que se ejecuta sobre una máquina virtual (JVM), lo que le da independencia respecto del sistema operativo subyacente [44]. Su madurez, su tipado estático y su ecosistema de bibliotecas lo han consolidado en el desarrollo de sistemas empresariales de larga vida [45].

Spring Boot es un marco de trabajo construido sobre el ecosistema Spring que reduce significativamente la configuración necesaria para levantar una aplicación: incorpora autoconfiguración, un servidor de aplicaciones embebido y un modelo de dependencias preempaquetadas que permite pasar de cero a un servicio operativo con poco código de infraestructura [46]. Ofrece además componentes para exponer el estado interno de la aplicación —métricas, verificaciones de salud, información de entorno— indispensables para operar un sistema distribuido sin acceso directo a un depurador [47].

Para el componente de clasificación se emplea Python, lenguaje interpretado de propósito general cuya principal ventaja en este contexto es el ecosistema de bibliotecas de análisis de datos y aprendizaje automático que ha crecido a su alrededor [48].

4.2.2. Interfaz de usuario

Angular es un marco de trabajo para construir aplicaciones web de página única (*single-page applications*), en las que la navegación ocurre en el navegador sin recargar el documento completo [49]. Está organizado en componentes reutilizables que encapsulan plantilla, estilos y lógica, e incorpora de forma nativa enrutamiento, gestión de formularios, cliente HTTP e inyección de dependencias, lo que evita ensamblar esas piezas a partir de bibliotecas sueltas [50]. El resultado de su compilación es un conjunto de archivos estáticos —HTML, CSS y JavaScript— que cualquier servidor web puede entregar, sin necesidad de ejecutar código del lado del servidor para la interfaz.

4.2.3. Contenedores

Un contenedor empaqueta una aplicación junto con todas sus dependencias en una unidad aislada que se ejecuta sobre el núcleo del sistema operativo anfitrión [51]. A diferencia de una máquina virtual, no incluye un sistema operativo completo, por lo que arranca en segundos y consume una fracción de los recursos. Su aporte principal es la reproducibilidad: la misma imagen se ejecuta de forma idéntica en la máquina de desarrollo y en el servidor de producción, lo que elimina toda una clase de fallos atribuibles a diferencias de entorno.

Los contenedores son, además, la unidad natural de despliegue de una arquitectura de microservicios: cada servicio se distribuye como una imagen propia, versionada e inmutable. Cuando el número de contenedores crece hasta volverse inmanejable manualmente, se recurre a plataformas de orquestación que automatizan su ciclo de vida, la recuperación ante fallos y el escalamiento [52]; a escalas menores, esa administración puede resolverse con guiones de despliegue, opción suficiente cuando el número de servicios es reducido y el equipo pequeño.

4.2.4. Proxy inverso

Un proxy inverso es un servidor que se coloca frente a uno o varios servicios y recibe en su nombre todas las peticiones del exterior, para después reenviarlas al componente que corresponda según la ruta solicitada [53]. Concentra en un solo punto responsabilidades que de otro modo habría que replicar en cada servicio: terminación de conexiones cifradas, balanceo de carga, entrega de contenido estático y control del tráfico entrante [54].

Su papel es doble en una aplicación web moderna. Por un lado entrega los archivos estáticos que componen la interfaz; por otro, enruta las peticiones dirigidas a la API hacia el servicio correspondiente. Desde el punto de vista del navegador todo proviene de un mismo origen, lo que simplifica el modelo de seguridad del cliente, mientras que del lado del servidor la separación en servicios permanece intacta.

4.2.5. Control de versiones

Git es un sistema de control de versiones distribuido que registra el historial completo de cambios de un proyecto y permite que varias personas trabajen en paralelo sobre la misma base de código mediante ramas que después se integran [55]. Su modelo distribuido implica que cada copia del repositorio contiene el historial íntegro, lo que aporta redundancia y permite trabajar sin conexión permanente al servidor central [56].

4.3. Seguridad aplicada al software

Una plataforma que administra denuncias maneja, por definición, información sensible: identifica a quien denuncia, a quien es denunciado y describe hechos que pueden tener consecuencias disciplinarias o jurídicas. Los mecanismos de esta sección son, por ello, requisitos del sistema y no características opcionales.

4.3.1. Autenticación basada en tokens

En una arquitectura distribuida, mantener la sesión del usuario en la memoria del servidor obliga a que todas sus peticiones lleguen siempre a la misma instancia, lo que limita el escalamiento

horizontal. La alternativa es la autenticación basada en tokens: el servicio de autenticación verifica las credenciales una sola vez y emite un token firmado que el cliente adjunta en cada petición posterior.

JSON Web Token (JWT) es el formato más extendido para ello. Se compone de tres partes —encabezado, carga útil y firma— codificadas y separadas por puntos. La carga útil contiene afirmaciones sobre el usuario, como su identidad, su rol y la fecha de expiración del token; la firma, calculada con una clave secreta, permite a cualquier servicio que la conozca verificar que el contenido no fue alterado [57]. Su propiedad más útil en un sistema distribuido es que la validación es local: un servicio puede autorizar una petición sin consultar al servicio emisor ni mantener estado de sesión compartido.

Esa misma propiedad tiene un costo: al ser autocontenido, un token válido no puede revocarse antes de su expiración sin introducir un mecanismo adicional de listas de revocación. De ahí que se recomiende emitirlos con tiempos de vida cortos.

4.3.2. Almacenamiento seguro de contraseñas

Las contraseñas nunca deben almacenarse en texto plano ni cifradas de forma reversible, pues quien obtenga acceso a la base de datos obtendría también las credenciales de todos los usuarios. La práctica correcta es guardar el resultado de una función resumen (*hash*) diseñada específicamente para este fin, que sea deliberadamente costosa de calcular y que incorpore un valor aleatorio distinto por usuario (*salt*), de modo que dos usuarios con la misma contraseña produzcan resúmenes diferentes y no puedan emplearse tablas precalculadas para revertirlos. La verificación en el inicio de sesión no consiste entonces en descifrar el valor almacenado, sino en recalcular el resumen de la contraseña recibida y compararlo con el registrado.

4.3.3. Control de acceso basado en roles

La autenticación resuelve quién es el usuario; la autorización, qué le está permitido hacer. El control de acceso basado en roles asigna a cada usuario uno o varios roles y asocia los permisos a esos roles y no a las personas, lo que simplifica la administración y reduce el riesgo de conceder privilegios por olvido. Debe aplicarse siguiendo el principio de menor privilegio: cada rol recibe únicamente los permisos indispensables para su función, y cada punto de acceso declara de forma explícita qué rol puede invocarlo. En una defensoría, esa distinción es también organizativa: quien recibe una queja no debe poder emitir una resolución, ni al contrario.

4.3.4. Intercambio de recursos entre orígenes

Los navegadores aplican la política de mismo origen, que impide a una página consultar recursos alojados en un dominio, protocolo o puerto distinto del suyo. El mecanismo CORS (*Cross-Origin Resource Sharing*) permite relajar esa restricción de forma controlada mediante encabezados HTTP con que el servidor declara qué orígenes acepta. La recomendación es enumerar explícitamente los orígenes permitidos en vez de autorizar cualquiera, y tener presente que el uso de un proxy inverso que sirva la interfaz y la API bajo un mismo dominio hace que el navegador ni siquiera active esta verificación.

4.3.5. Cifrado en tránsito

Toda la información que viaja entre el navegador y el servidor debe protegerse con TLS (*Transport Layer Security*), protocolo que establece un canal cifrado y autenticado sobre una conexión de red no confiable. Su versión 1.3 simplificó el proceso de negociación, redujo la latencia de establecimiento de la conexión y eliminó los algoritmos criptográficos considerados inseguros en versiones previas [58]. Sin este cifrado, las credenciales, los tokens de sesión y el contenido íntegro de una denuncia viajarían legibles para cualquier intermediario de la red.

4.4. Problema de clasificación en el contexto de la DDP

El problema central de este proyecto es automatizar la decisión de admisibilidad de una queja ante la Defensoría de los Derechos Politécnicos, decisión que hoy depende del criterio manual del personal de primer contacto y genera inconsistencias operativas. Su punto de partida es el marco legal del Acuerdo publicado en la Gaceta Politécnica, Número Extraordinario 622 [2], donde se establece a la DDP como el órgano encargado de promover, proteger y defender los derechos de la comunidad frente a actos u omisiones de las autoridades del Instituto. Comprender los límites normativos de la competencia institucional es, por tanto, el primer paso para diseñar un clasificador que replique la lógica jurídica con precisión.

El Artículo 3.º de ese Acuerdo establece que la Defensoría es competente para conocer quejas relacionadas con presuntas violaciones a los derechos de la comunidad politécnica. El Artículo 4.º, en cambio, delimita las materias en las que no lo es:

1. Afectaciones de carácter colectivo.
2. Resoluciones disciplinarias.
3. Derechos de naturaleza laboral.
4. Evaluaciones académicas de profesores, comisiones dictaminadoras o consejos técnicos consultivos escolares, salvo que en dicha evaluación se violen notoriamente los derechos de los interesados.
5. Procedimientos y resoluciones instaurados por el Órgano Interno de Control en el Instituto.

El procedimiento de recepción descrito en el Artículo 13 obliga al personal de la Defensoría a determinar, en primera instancia, si la queja es competente o incompetente. Esa decisión, en apariencia sencilla, resulta en la práctica altamente ambigua por la forma en que están redactados los hechos, y su dependencia del criterio manual genera tres conflictos críticos:

1. El consumo excesivo de tiempo administrativo.
2. La variabilidad de interpretaciones ante casos similares.
3. La saturación operativa por el incremento constante de solicitudes.

El problema se define entonces como una tarea de Procesamiento de Lenguaje Natural (PLN) y clasificación supervisada, en la que el sistema debe aprender los patrones lingüísticos y normativos que separan una queja competente de una incompetente.

4.4.1. Taxonomía de denuncias

Las denuncias que recibe la DDP no se clasifican solo por su contenido textual, sino por una estructura jerárquica de validación normativa que el sistema debe aprender a reconocer; establecer una taxonomía clara es, por ello, condición previa para construir cualquier modelo. Transformar una narrativa jurídica en un conjunto de datos procesable por algoritmos de aprendizaje automático exige organizar el conocimiento normativo de forma sistemática. En el contexto de la Defensoría, esa taxonomía descansa en dos reglas críticas:

1. **Pertenencia a la comunidad y jerarquía.** Define si las personas involucradas pertenecen a la comunidad politécnica. El sistema debe identificar al quejoso junto con su estatus institucional y a la autoridad a quien se imputa la violación. Sin esa validación de identidad, la queja carece de base para ser procesada.
2. **Exclusiones del Artículo 4.º** Constituyen el núcleo del clasificador. Las denuncias se categorizan según la presencia de los cinco criterios de exclusión señalados antes, criterios que el sistema traduce en siete campos binarios — atributos jurídicos estructurados — que operacionalizan los siguientes contrastes normativos:
 - Naturaleza laboral frente a derechos politécnicos.
 - Evaluación académica pura frente a vulneración de derechos dentro de la evaluación.
 - Afectación individual frente a colectiva.
 - Resolución disciplinaria frente a acto de autoridad.
 - Procedimientos del Órgano Interno de Control frente a acto u omisión de autoridad institucional.

Esta estructuración logra que la clasificación no dependa únicamente del texto libre, sino de las reglas establecidas en el Artículo 4.º Al término del proceso taxonómico, el sistema asigna a cada expediente una etiqueta binaria definitiva —Competente o No Competente, esta última con canalización a otra instancia o desechamiento—, que constituye la variable de salida de los modelos de clasificación supervisada.

4.4.2. Clasificación binaria y lógica de admisión

Definida la taxonomía normativa, el siguiente paso es determinar cómo producirá el clasificador su veredicto. El sistema adopta un esquema de clasificación binaria, decisión fundamentada en el Artículo 13 del Acuerdo Número Extraordinario 622. A diferencia de los sistemas de categorización temática —que discriminan entre dominios como deportes, política o finanzas—, el clasificador de la DDP emite un veredicto dicotómico: Competente o No Competente. Esta reducción a dos clases responde a la lógica de admisión procesal por tres razones:

1. **Naturaleza del flujo administrativo.** Dentro de la DDP, el primer contacto funciona como una compuerta. No existe un estado intermedio: la queja se admite para investigación o se desecha y canaliza a otra instancia. La clasificación binaria representa fielmente ese proceso de toma de decisiones.
2. **Optimización del entrenamiento.** Desde la perspectiva del aprendizaje automático, concentrar la muestra en dos etiquetas produce mayor densidad de datos por categoría [59]. Dado que el volumen de expedientes históricos etiquetados es finito, un modelo binario resulta más robusto y menos propenso al error que uno multiclase que intentara identificar además el tipo específico de exclusión —laboral, académica o disciplinaria—.

3. **Reducción del falso negativo.** La lógica de admisión prioriza la protección de los derechos de la comunidad. Al plantear el problema como binario es posible ajustar el umbral de decisión —en especial en algoritmos como SVM— para minimizar los falsos negativos y asegurar que ninguna queja potencialmente competente se descarte por una clasificación errónea.

En consecuencia, si el modelo predice la clase No Competente, el sistema asiste al personal sugiriendo la fundamentación legal derivada del Artículo 4.º Si la predicción es Competente, el expediente fluye de forma automática hacia la etapa de radicación en la Subdefensoría, con lo que se cumple la meta de reducir los tiempos de respuesta institucional.

4.4.3. Ciclo de vida de los datos en el aprendizaje supervisado

Para que la clasificación binaria opere de manera confiable, la narrativa jurídica del quejoso debe recorrer un flujo técnico que la convierta en una señal numérica apta para el aprendizaje automático. El ciclo de vida de los datos organiza en seis etapas ese puente entre el texto libre y la decisión del modelo, y asegura la trazabilidad e integridad de la información jurídica en cada fase:

1. **Captura de datos.** El ciclo inicia en la interfaz de registro, donde el quejoso proporciona de forma estructurada su identidad institucional y, de forma no estructurada, el relato de los hechos. Esta etapa asegura que la entrada cumpla los requisitos mínimos de admisión del Artículo 13.
2. **Preprocesamiento y limpieza.** La narrativa libre se somete a técnicas de PLN: se elimina el ruido, se normaliza el texto y se identifican los términos clave que corresponden a las exclusiones normativas del Artículo 4.º
3. **Vectorización y enriquecimiento.** El texto se convierte en representaciones vectoriales (*embeddings*). En paralelo, el sistema inyecta los atributos jurídicos binarios extraídos del expediente y aplica el factor de amplificación α para dar peso a la señal normativa frente a la semántica.
4. **Entrenamiento y ajuste.** Con un conjunto de datos histórico previamente etiquetado, el modelo aprende a identificar las fronteras de decisión legal. Se emplean técnicas de validación cruzada para medir la robustez de la generalización.
5. **Inferencia y sugerencia.** Ante una queja nueva, el modelo procesa los vectores y genera una predicción de competencia acompañada de un nivel de probabilidad. El sistema asiste al personal mostrando el veredicto sugerido y la posible fundamentación legal.
6. **Retroalimentación.** El ciclo se cierra cuando el personal jurídico valida o corrige la sugerencia del modelo. Esas decisiones se reincorporan al repositorio de datos para el reentrenamiento futuro, de modo que el clasificador evolucione conforme cambian los criterios institucionales.

4.5. Fundamentos de Procesamiento de Lenguaje Natural

El Procesamiento de Lenguaje Natural es la rama de la inteligencia artificial que estudia cómo dotar a las computadoras de la capacidad de leer, interpretar y generar texto humano [60]. Su importancia radica en que el lenguaje natural —a diferencia de los datos numéricos o tabulares—

es ambiguo por naturaleza, depende del contexto y está lleno de matices que una máquina no puede procesar de forma directa. Para que un algoritmo razone sobre un texto, este debe transformarse primero en una representación matemática que preserve su significado.

En este proyecto el PLN cumple un rol central: las quejas que recibe la DDP llegan como narrativas libres, redactadas en lenguaje coloquial o semijurídico por personas sin formación legal. Ello introduce desafíos ausentes en dominios más estandarizados: el vocabulario especializado varía entre quejosos, la estructura argumentativa suele ser implícita y la clasificación jurídica correcta depende no solo de las palabras empleadas, sino de la relación normativa entre los hechos descritos y los artículos del Acuerdo 622. Dos quejas léxicamente parecidas pueden tener, por eso, clasificaciones opuestas según el contexto institucional que las rodea.

Para atender esos desafíos el sistema requiere dos tipos de herramientas de PLN, que se desarrollan en las subsecciones siguientes: el fundamento matemático que permite representar documentos como vectores comparables y las transformaciones lingüísticas que preparan y limpian el texto antes de aplicar ese fundamento.

4.5.1. Álgebra lineal en espacios vectoriales de alta dimensión

Para que una computadora compare documentos, determine cuáles se parecen o trace una frontera que separe categorías, esos documentos deben existir antes como objetos matemáticos. El álgebra lineal provee ese lenguaje: permite representar cualquier texto como un vector —una lista ordenada de números— y medir después las relaciones entre vectores con precisión aritmética. Esta representación es el fundamento sobre el que operan todos los vectorizadores y clasificadores del sistema; sin ella no existiría mecanismo alguno para que el modelo aprenda qué quejas se parecen entre sí ni cómo trazar la línea que separa las competentes de las incompetentes.

La idea central es que cualquier documento puede representarse como un punto en un espacio vectorial de alta dimensión $\mathbb{R}^d$, donde d es el número de características que lo describen. Documentos semánticamente similares —por ejemplo, dos quejas sobre acoso académico— ocupan regiones cercanas de ese espacio, mientras que documentos disímiles —una queja laboral y una queja por evaluación— quedan alejados. La similitud coseno formaliza esa noción de cercanía entre dos documentos $\mathbf{u}$ y $\mathbf{v}$:

$$\text{sim}(\mathbf{u}, \mathbf{v}) = \frac{\mathbf{u} \cdot \mathbf{v}}{\|\mathbf{u}\| \|\mathbf{v}\|} = \frac{\sum_{i=1}^d u_i v_i}{\sqrt{\sum_{i=1}^d u_i^2} \sqrt{\sum_{i=1}^d v_i^2}} \quad (4.1)$$

donde $\text{sim} \in [-1, 1]$. Un valor cercano a 1 indica alta similitud semántica; esta métrica es la base del razonamiento por proximidad que subyace a todos los vectorizadores del sistema.

El número de dimensiones con que se representan los documentos no es arbitrario. En este proyecto se trabaja con dos:

- **300 dimensiones**, correspondientes a spaCy. Sus vectores fueron entrenados por Explosion AI con word2vec, algoritmo que Mikolov et al. publicaron originalmente con vectores de 300 dimensiones [61], medida que terminó por convertirse en un estándar de la industria.
- **768 dimensiones**, correspondientes a Sentence-BERT. Ese número proviene de la arquitectura BERT base, cuyas capas de atención emplean una dimensión oculta de 768; al

construir SBERT sobre BERT, el vector de salida del *pooling* final hereda esa dimensión [13].

A mayor dimensión, el modelo captura matices semánticos más finos —por ejemplo, distinguir entre “no se me permitió acceder” y “accedí sin autorización”—, pero también exige más capacidad computacional y más datos para evitar la maldición de la dimensionalidad: conforme crece d , el volumen del espacio aumenta de forma exponencial y los datos se vuelven dispersos, lo que dificulta encontrar vecinos significativos. Sobre estos espacios el sistema ejecuta cuatro operaciones elementales del álgebra lineal:

- **Suma de vectores:** combina las representaciones de tokens individuales en una representación de documento.
- **Promedio ponderado:** base del vectorizador de spaCy.
- **Producto punto y norma:** los emplea internamente la SVM para calcular márgenes de separación.
- **Concatenación horizontal:** operación central de la arquitectura híbrida, que une los *embeddings* de texto con los atributos jurídicos.

Para que estas operaciones produzcan vectores con significado real, el texto de entrada debe estar limpio y estandarizado. Ningún modelo puede extraer un vector confiable de una queja plagada de errores tipográficos, abreviaturas inconsistentes o sinónimos tratados como palabras distintas. De ahí la necesidad del preprocesamiento que se describe a continuación.

4.5.2. Preprocesamiento de la queja

Antes de que el álgebra lineal opere sobre el texto, la narrativa libre del quejoso pasa por una cadena de transformaciones que reducen el ruido y estandarizan la representación lingüística. Este preprocesamiento resulta especialmente relevante en textos jurídicos, donde las variaciones ortográficas, los errores tipográficos o los sinónimos coloquiales de términos normativos afectan directamente la calidad de los *embeddings*. Las transformaciones se aplican en el siguiente orden.

Tokenización. Segmenta el texto continuo en unidades mínimas de significado, o tokens [60]. spaCy realiza una tokenización sensible al contexto que distingue, por ejemplo, contracciones y signos de puntuación:

"me negó el acceso" → [me, negó, el, acceso]

Lematización. Reduce cada token a su forma canónica, o lema, y elimina las variaciones morfológicas [60]:

negaron → negar, discriminado → discriminar, estudiantiles → estudiantil

Filtrado de palabras vacías. Las palabras vacías (*stop words*) son términos de alta frecuencia y bajo contenido semántico: artículos, preposiciones y conjunciones. En textos jurídicos, sin embargo, algunas palabras que en otros contextos serían prescindibles —como “no”, “sin” o “nunca”— pueden resultar jurídicamente decisivas. spaCy permite personalizar esa lista para el dominio específico de la Defensoría.

Análisis morfosintáctico con spaCy. El modelo `es_core_news_lg` realiza, además de la tokenización y la lematización, un análisis morfosintáctico completo que asigna a cada token su categoría gramatical (*POS tagging*) y sus dependencias sintácticas [62]. Ello permite identificar, por ejemplo, que en la oración “El director me negó el acceso al laboratorio” el sujeto es *director* (la autoridad), el verbo es *negar* (el acto u omisión) y el complemento es *acceso al laboratorio* (el derecho afectado). Esta información estructural enriquece la representación semántica más allá del léxico superficial y sienta las bases de la etapa de vectorización.

4.6. Representación vectorial del texto

Normalizado el texto por el preprocesamiento, el siguiente desafío es convertirlo en representaciones numéricas que los algoritmos de aprendizaje automático puedan procesar. La representación vectorial es, por tanto, el puente entre el lenguaje natural y los clasificadores. A continuación se presenta la evolución de estas técnicas, desde los modelos estadísticos hasta los modelos de lenguaje contextuales, mostrando en cada etapa por qué los enfoques más sencillos resultan insuficientes para el dominio jurídico de la DDP.

4.6.1. Modelos de bolsa de palabras y TF-IDF

Los modelos estadísticos de representación textual son el punto de partida histórico de la vectorización y permiten comprender, por contraste, las limitaciones que motivan el uso de *embeddings* semánticos. Los modelos de bolsa de palabras (*Bag of Words*, BoW) representan un documento como un vector de frecuencias de términos sobre el vocabulario total del corpus: cada palabra se convierte en una dimensión separada del espacio vectorial [59], de modo que un conjunto de texto con n palabras produce un espacio de n dimensiones, y la posición de un documento en él queda determinada por cuántas veces aparece cada término. Formalmente:

$$\mathbf{v}_{\text{BoW}}(d) = [f(t_1, d), f(t_2, d), \dots, f(t_V, d)] \quad (4.2)$$

donde $f(t_i, d)$ es la frecuencia del término t_i en el documento d y V es el tamaño del vocabulario. TF-IDF (*Term Frequency – Inverse Document Frequency*) refina este enfoque al ponderar cada término según su frecuencia en el documento y su rareza en el corpus [59]:

$$\text{tf-idf}(t, d, D) = \underbrace{\frac{f(t, d)}{\sum_{t'} f(t', d)}}_{\text{TF}} \times \underbrace{\log \frac{|D|}{|\{d' \in D : t \in d'\}|}}_{\text{IDF}} \quad (4.3)$$

Limitaciones críticas para el dominio jurídico. Aunque son eficientes en términos computacionales, estos modelos resultan inadecuados para clasificar las quejas de la DDP por dos razones:

1. **Pérdida del orden y del contexto.** Las frases “el director negó el acceso” y “el acceso negó al director” generan el mismo vector pese a significar cosas radicalmente distintas.
2. **Sesgo léxico superficial.** El término “calificación” presenta alta frecuencia tanto en quejas por evaluación académica (No Competente) como en quejas por extorsión académica (Competente), lo que vuelve imposible distinguirlas correctamente.

Estos modelos se exponen aquí como antecedente necesario para contextualizar la superioridad de los enfoques basados en *embeddings* que emplea el sistema.

4.6.2. Word embeddings estáticos: spaCy (es_core_news_lg)

Dado que los modelos estadísticos pierden el contexto semántico, la siguiente generación de técnicas representa cada palabra como un vector denso aprendido de grandes corpus. Esta representación, conocida como *embedding* estático, resuelve la limitación semántica de BoW al situar palabras relacionadas —como *acoso* y *hostigamiento*— en regiones cercanas del espacio vectorial. Los *embeddings* estáticos representan cada palabra como un vector denso en $\mathbb{R}^{300}$, entrenado sobre grandes corpus en español mediante el algoritmo word2vec [61].

El modelo `es_core_news_lg` de spaCy [62] fue entrenado sobre el corpus *Spanish Web Crawl* y genera vectores de 300 dimensiones por token. La representación de un documento completo se obtiene promediando los vectores de los tokens informativos:

$$\mathbf{v}_{\text{spaCy}}(d) = \frac{1}{|T|} \sum_{t \in T} \mathbf{w}_t, \quad T = \{t \in d \mid \text{has_vector}(t) \wedge \neg \text{is_stop}(t)\} \quad (4.4)$$

donde $\mathbf{w}_t \in \mathbb{R}^{300}$ es el vector del token t y T es el conjunto de tokens informativos, es decir, aquellos que cuentan con vector disponible y no son palabras vacías.

Al promediar todos los vectores del documento, no obstante, se pierden el orden sintáctico y el contexto local. Las frases “el profesor me acosó condicionando la calificación” y “el profesor condicionó la calificación sin acosar” generarían vectores cercanos pese a tener clasificaciones jurídicas distintas. Esta limitación motiva el uso de modelos contextuales como Sentence-BERT.

4.6.3. Sentence transformers: SBERT paraphrase-multilingual-mpnet-base-v2

Para superar la insensibilidad al orden y al contexto de los *embeddings* estáticos se requiere un modelo que genere una representación de toda la oración considerando las relaciones entre sus partes. Sentence-BERT (SBERT) satisface ese requisito y representa el estado del arte en codificación semántica de textos [13]: a diferencia de los *embeddings* estáticos, genera un único vector para la oración completa y captura las relaciones entre palabras a lo largo de todo el texto mediante la arquitectura *Transformer*.

Arquitectura Transformer. El bloque central del *Transformer* es el mecanismo de atención multicabeza (*multi-head self-attention*), que calcula para cada token su relevancia en función de todos los demás tokens del documento:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V \quad (4.5)$$

donde Q , K y V son las matrices de consulta, clave y valor derivadas de los *embeddings* de entrada, y d_k es la dimensión de la clave [63]. Este mecanismo permite que el modelo asigne mayor peso a los tokens contextualmente relevantes. Por ejemplo, en la queja “me reprobó porque no acepté salir con él”, la atención aprende a conectar *reprobó* con *acepté salir* e identifica que la relación causal entre ambos constituye un acto de acoso y no una evaluación académica pura.

Entrenamiento contrastivo de SBERT. SBERT extiende BERT con una capa de *pooling* sobre los estados ocultos y se entrena mediante aprendizaje contrastivo con pares de oraciones semánticamente equivalentes y no equivalentes:

$$\mathcal{L}_{\text{contrastiva}} = \sum_{(i,j)} (\text{sim}(\mathbf{e}_i, \mathbf{e}_j) - y_{ij})^2 \quad (4.6)$$

donde $y_{ij} = 1$ si las oraciones i y j son semánticamente equivalentes y $y_{ij} = 0$ en caso contrario.

Modelo empleado. El modelo paraphrase-multilingual-mpnet-base-v2 fue entrenado sobre más de 50 idiomas, entre ellos el español. Genera *embeddings* de 768 dimensiones por documento [64] y ha mostrado un desempeño superior al de los promedios de *word embeddings* en tareas de clasificación de texto corto y mediano. Esa capacidad resulta crítica para distinguir los casos límite de la DDP, donde el acto jurídicamente relevante puede estar expresado en una sola oración dentro de un párrafo extenso.

4.7. Ingeniería de características híbridas

Aunque los *embeddings* semánticos capturan el significado del texto, los modelos puramente textuales pueden aprender correlaciones léxicas espurias en lugar de la lógica jurídica subyacente. Por ello, una de las contribuciones centrales de este sistema es la arquitectura híbrida que combina la representación semántica con conocimiento normativo estructurado. Esa arquitectura opera en dos pasos: primero codifica los criterios del Artículo 4.º como variables binarias y después las fusiona con el *embedding* textual mediante una concatenación amplificada que equilibra ambas señales.

4.7.1. Codificación de variables normativas

Los cinco criterios de incompetencia del Artículo 4.º se traducen en siete variables binarias — los atributos jurídicos — que codifican el análisis normativo de cada queja de forma estructurada. Esta codificación es el mecanismo que permite al clasificador razonar con base en la ley y no solo en la semántica del texto. Las variables y su descripción normativa se presentan en la Tabla 4.2.

Tabla 4.2: Atributos jurídicos estructurados derivados del Artículo 4.º

#	Variable	Descripción normativa
1	es_conflicto_laboral_art4_I	El fondo del conflicto es una relación laboral o sindical: salarios, plazas, escalafón o prestaciones.
2	es_evaluacion_academica_pura_art4_II	La queja versa únicamente sobre la valoración del aprendizaje por parte de un profesor o de una comisión.
3	hay_vulneracion_notoria_en_evaluacion	Existe una violación de derechos dentro de un proceso evaluativo: extorsión, acoso o discriminación.
4	es_resolucion_disciplinaria_art4_III	La queja impugna una resolución disciplinaria ya emitida.
5	es_afectacion_colectiva_art4_IV	La afectación descrita es grupal y no individual.
6	involucra_oic_art4_V	El asunto está siendo conocido por el Órgano Interno de Control.
7	hay_acto_u_omision_autoridad	Existe un acto u omisión de una autoridad institucional que vulnera derechos politécnicos.

Estas variables se representan como un vector $\mathbf{f} \in \{0, 1\}^7$ y se determinan a partir del análisis del expediente, ya sea de forma manual durante la construcción del corpus o de forma automática,

en una etapa futura, mediante un modelo extractor. La lógica de consistencia entre las variables y la clase asignada es la siguiente:

- Si la clase es No Competente por razón laboral, $\text{es_conflicto_laboral_art4_I} = 1$ y el resto de las variables $= 0$.
- Si la clase es Competente, $\text{hay_acto_u_omision_autoridad} = 1$ y todas las fracciones del Artículo 4.º $= 0$.
- La variable 3 ($\text{hay_vulneracion_notoria}$) solo puede valer 1 cuando la variable 2 ($\text{es_evaluacion_academica_pura}$) también vale 1 y la clase es Competente; así se modela la excepción prevista en la fracción IV del Artículo 4.º

4.7.2. Concatenación y factor de amplificación (α)

Codificadas las variables normativas, es necesario fusionarlas con el *embedding* semántico de manera que ambas señales contribuyan de forma equitativa a la decisión del clasificador. La unión se realiza mediante concatenación horizontal junto con un factor de amplificación α que compensa la diferencia de escala entre los vectores:

$$\mathbf{x}_{\text{final}} = [\mathbf{e} \parallel \alpha \cdot \mathbf{f}] \in \mathbb{R}^{d+7} \quad (4.7)$$

donde $d \in \{300, 768\}$ según el vectorizador empleado.

Justificación matemática del factor α . Sin amplificación ($\alpha = 1$), los siete atributos jurídicos quedan opacados por las d dimensiones del *embedding*, cuya norma promedio es bastante mayor. Para ilustrarlo, considérese la contribución relativa de cada componente a la norma del vector concatenado:

$$\frac{\|\alpha \cdot \mathbf{f}\|}{\|\mathbf{e}\| + \|\alpha \cdot \mathbf{f}\|} = \frac{\alpha\sqrt{7}}{\|\mathbf{e}\| + \alpha\sqrt{7}} \quad (4.8)$$

Con $\|\mathbf{e}\| \approx 15$, valor típico para *embeddings* de Sentence-BERT normalizados, y $\alpha = 1$:

$$\frac{1 \cdot \sqrt{7}}{15 + \sqrt{7}} \approx \frac{2,65}{17,65} \approx 15 \%$$

Con $\alpha = 3$:

$$\frac{3\sqrt{7}}{15 + 3\sqrt{7}} \approx \frac{7,94}{22,94} \approx 35 \%$$

El valor $\alpha = 3,0$ se seleccionó de forma experimental como el punto de equilibrio en el que la señal jurídica pesa lo suficiente para influir en las fronteras de decisión del clasificador sin dominar por completo la información semántica del texto.

4.7.3. Simulación de producción: entrenamiento con metadatos e inferencia en modo autónomo

La arquitectura híbrida introduce una asimetría importante entre el escenario de entrenamiento y el de producción, que debe modelarse de forma explícita para evitar que el sistema resulte sobreoptimista en sus métricas de evaluación. Esa asimetría surge porque, durante la construcción del corpus, el personal jurídico anota manualmente los siete atributos booleanos del Artículo 4.º, mientras que en producción el sistema recibe únicamente el relato libre del quejoso, sin anotación previa. La asimetría se modela mediante dos modos de operación:

1. **Modo asistido.** El personal de primer contacto completa el formulario con los atributos jurídicos mientras atiende al quejoso. El clasificador opera con el vector completo $\mathbf{x}_{\text{final}} = [\mathbf{e} \parallel \alpha \cdot \mathbf{f}]$ y aprovecha tanto la semántica del texto como la señal normativa estructurada.
2. **Modo autónomo.** Los atributos jurídicos se fijan en cero ($\mathbf{f} = \mathbf{0}$), de forma que el clasificador basa su veredicto exclusivamente en la representación semántica del texto: $\mathbf{x}_{\text{final}} = [\mathbf{e} \parallel \mathbf{0}]$. Este modo representa el escenario real de producción y constituye la condición de estrés bajo la cual se evalúa la robustez de los modelos.

La distinción entre ambos modos tiene implicaciones directas para la selección del clasificador óptimo: un modelo que dependa en exceso de la señal normativa amplificada puede exhibir métricas perfectas durante el entrenamiento y degradarse de forma significativa en producción, cuando esa señal no está disponible. Por esa razón, el criterio de selección del modelo final privilegia el desempeño en modo autónomo sobre el desempeño con metadatos.

4.8. Modelos de aprendizaje automático evaluados

Definido el vector de características híbridas, el siguiente paso es elegir el algoritmo que mejor aprenda a partir de él las fronteras de decisión legal. Para garantizar la selección del clasificador óptimo para el dominio jurídico de la DDP se evaluaron cuatro configuraciones de aprendizaje supervisado con paradigmas distintos de construcción de fronteras. Cada modelo aporta una perspectiva diferente sobre los datos, y su comparación permite identificar cuál se adapta mejor a la distribución de las quejas en el espacio vectorial.

4.8.1. Máquinas de soporte vectorial (SVM)

Las máquinas de soporte vectorial (*Support Vector Machines*, SVM) son algoritmos de clasificación que buscan el hiperplano de máximo margen capaz de separar las clases en el espacio de características, lo que las hace especialmente robustas ante casos límite como las quejas situadas en la frontera entre competente e incompetente [11]. Dado un conjunto de entrenamiento $\{(\mathbf{x}_i, y_i)\}_{i=1}^n$ con $y_i \in \{-1, +1\}$, la SVM resuelve el problema de optimización:

$$\begin{aligned} \min_{\mathbf{w}, b, \xi} \quad & \frac{1}{2} \|\mathbf{w}\|^2 + C \sum_{i=1}^n \xi_i \\ \text{s.a.} \quad & y_i(\mathbf{w}^\top \mathbf{x}_i + b) \geq 1 - \xi_i, \quad \xi_i \geq 0 \end{aligned} \tag{4.9}$$

donde C es el parámetro de regularización que controla la tolerancia al error de clasificación —con valor $C = 0,5$ en este sistema— y ξ son las variables de holgura que permiten una clasificación blanda.

Kernel RBF. Cuando los datos no son linealmente separables en el espacio original, la SVM los proyecta a un espacio de mayor dimensión mediante una función de kernel. El kernel de base radial (*Radial Basis Function*, RBF) que emplea este sistema calcula la similitud entre dos puntos como:

$$K(\mathbf{x}_i, \mathbf{x}_j) = \exp(-\gamma \|\mathbf{x}_i - \mathbf{x}_j\|^2) \quad (4.10)$$

donde γ controla el radio de influencia de cada vector de soporte. El kernel RBF resulta especialmente adecuado para *embeddings* de alta dimensión porque puede construir fronteras de decisión no lineales arbitrariamente complejas, adaptándose a la distribución de los casos jurídicos en el espacio semántico [11].

Ventaja en el dominio jurídico. La SVM con kernel RBF maximiza el margen de separación entre las clases, lo que le confiere robustez natural ante casos límite: las quejas que caen en la frontera entre Competente y No Competente se clasifican según el hiperplano de máximo margen y no según la densidad local de puntos. Esa ventaja geométrica está condicionada, sin embargo, a que la señal de entrada sea suficientemente discriminativa; cuando los atributos jurídicos no están presentes, la eficacia del clasificador depende por completo de la calidad del *embedding* semántico.

4.8.2. Regresión logística

A diferencia de la SVM, que construye una frontera geométrica de máximo margen, la regresión logística es un modelo lineal que estima directamente la probabilidad de pertenencia a cada clase. Ello le confiere una interpretabilidad valiosa en el contexto jurídico, donde la explicabilidad de la decisión puede exigirse institucionalmente. La probabilidad de pertenencia a la clase positiva se estima mediante la función logística o sigmoide:

$$P(y = 1 | \mathbf{x}) = \sigma(\mathbf{w}^\top \mathbf{x} + b) = \frac{1}{1 + e^{-(\mathbf{w}^\top \mathbf{x} + b)}} \quad (4.11)$$

Los parámetros $\mathbf{w}$ y b se estiman minimizando la función de pérdida de entropía cruzada binaria:

$$\mathcal{L}(\mathbf{w}, b) = -\frac{1}{n} \sum_{i=1}^n \left[y_i \log \hat{p}_i + (1 - y_i) \log(1 - \hat{p}_i) \right] + \lambda \|\mathbf{w}\|^2 \quad (4.12)$$

donde λ es el término de regularización L2. La configuración empleada en este sistema es `max_iter=1000` con $C = 0.3$. Los coeficientes w_j son además directamente interpretables: un coeficiente positivo asociado a una dimensión del *embedding* indica que ese componente favorece la clasificación como Competente, lo que facilita la auditoría del modelo.

4.8.3. Ensamble de árboles: Random Forest

Donde la SVM y la regresión logística construyen una sola frontera de decisión, Random Forest resuelve el problema agregando múltiples árboles de decisión entrenados sobre subconjuntos aleatorios de los datos. Esta estrategia de ensamble reduce la varianza de la predicción individual y confiere al modelo una notable capacidad de generalización [12]. Formalmente:

$$\hat{y}_{\text{RF}}(\mathbf{x}) = \frac{1}{B} \sum_{b=1}^B T_b(\mathbf{x}) \quad (4.13)$$

donde B es el número de árboles — $B = 300$ en este sistema— y T_b es el b -ésimo árbol entrenado sobre una muestra *bootstrap*. Los árboles particionan el espacio de características con cortes ortogonales a los ejes y construyen de forma recursiva regiones de decisión rectangulares. Aunque en espacios de dimensión muy alta esa estrategia puede ser menos eficiente que el hiperplano de máximo margen de la SVM, el promedio de un número grande de árboles con submuestreo aleatorio de características reduce de forma significativa la varianza y resulta especialmente robusto cuando la representación semántica del texto no está apoyada por atributos estructurados.

4.8.4. Ensamble por votación suave

Dado que cada clasificador individual tiene fortalezas complementarias, una estrategia de ensamble que combine sus predicciones puede alcanzar un desempeño superior al de cualquiera por separado. La votación suave (*soft voting*) aprovecha esa propiedad promediando las probabilidades predichas por múltiples clasificadores base para obtener una predicción final más estable [65]:

$$\hat{y} = \arg \max_c \frac{1}{M} \sum_{m=1}^M p_m(c | \mathbf{x}) \quad (4.14)$$

donde M es el número de modelos base y $p_m(c | \mathbf{x})$ es la probabilidad que el modelo m asigna a la clase c . A diferencia de la votación dura, que solo cuenta votos de clase, la votación suave aprovecha la magnitud de la confianza de cada clasificador, lo que suele producir predicciones mejor calibradas cuando las tasas de error de los modelos base no están correlacionadas.

En este sistema el ensamble integra SVM-RBF, regresión logística y Random Forest. La motivación es que cada uno aporta algo distinto: la SVM maximiza el margen geométrico, la regresión logística produce probabilidades linealmente calibradas y Random Forest captura fronteras no lineales por partición del espacio. Si sus errores no están correlacionados, el ensamble reduce la varianza de la predicción respecto a cualquiera de los modelos base. Para que `VotingClassifier` opere en modo `voting="soft"`, todos los estimadores deben exponer el método `predict_proba`; dado que `SVC` no lo implementa de forma nativa, se envuelve en `CalibratedClassifierCV`, que estima probabilidades mediante validación cruzada interna sobre el conjunto de entrenamiento [66].

4.9. Estrategias de entrenamiento y validación robusta

La selección del mejor clasificador no puede basarse en métricas obtenidas sobre los mismos datos con que se entrenó, pues ello produciría una estimación optimista del desempeño real. Las estrategias de entrenamiento y validación robusta garantizan que las métricas reportadas reflejen la capacidad de generalización del sistema y no sean un artefacto de una partición particular de los datos.

4.9.1. Validación cruzada K -fold estratificada

La validación cruzada K -fold es la estrategia central para estimar el desempeño de los modelos de manera confiable: divide el conjunto de datos en K particiones de tamaño aproximadamente igual y rota el conjunto de evaluación, de modo que todos los datos se usen para prueba exactamente una vez. En cada iteración, $K - 1$ particiones sirven para entrenamiento y la restante para evaluación, lo que produce una estimación promedio del desempeño:

$$F1_{CV} = \frac{1}{K} \sum_{k=1}^K F1_k \quad (4.15)$$

La variante estratificada garantiza que la proporción de clases se mantenga en cada pliegue, condición crítica ante corpus con posible desbalance entre quejas competentes e incompetentes [66]. En este proyecto se empleó $K = 5$, lo que significa que en cada iteración el 80 % de los datos se usa para entrenamiento y el 20 % restante para evaluación. Además del promedio, la desviación estándar entre pliegues (σ_{CV}) se emplea como medida de estabilidad del modelo:

$$\sigma_{CV} = \sqrt{\frac{1}{K} \sum_{k=1}^K (F1_k - F1_{CV})^2} \quad (4.16)$$

Un σ_{CV} cercano a cero indica que el modelo no es sensible a la partición específica de los datos, condición deseable para un sistema de apoyo en producción.

Nota de implementación. En este proyecto la validación cruzada con $K = 5$ se aplica exclusivamente sobre el conjunto de entrenamiento —el 70 % del corpus, es decir, 367 casos— y no sobre el conjunto de datos completo, de modo que cada pliegue evalúa alrededor de 73 casos internos del entrenamiento. Esta validación se complementa con un conjunto *holdout* independiente del 15 % (79 casos), aislado por completo durante el entrenamiento y la selección de hiperparámetros, cuyo F1 ponderado —evaluado en modo autónomo, sin atributos jurídicos— constituye el criterio definitivo de selección del modelo.

4.9.2. Evaluación mediante métricas de desempeño

Las métricas de evaluación deben reflejar las prioridades institucionales de la DDP y no únicamente la exactitud global del modelo. Se emplean, por tanto, métricas que distinguen entre los distintos tipos de error y asignan mayor peso al de mayor costo institucional: clasificar como incompetente una queja que sí debía atenderse. Para comprenderlas conviene establecer primero los cuatro resultados posibles de una clasificación binaria, organizados en la matriz de confusión:

- **Verdadero positivo (VP):** la queja es competente y el modelo la clasifica como competente.
- **Verdadero negativo (VN):** la queja es incompetente y el modelo la clasifica como incompetente.
- **Falso positivo (FP):** la queja es incompetente pero el modelo la clasifica como competente.

- **Falso negativo (FN):** la queja es competente pero el modelo la clasifica como incompetente. En el contexto de la DDP este es el error más grave, pues implica descartar una queja que debía ser atendida.

Exactitud. La exactitud (*accuracy*) mide la proporción de predicciones correctas sobre el total de instancias:

$$\text{Exactitud} = \frac{VP + VN}{VP + VN + FP + FN} \quad (4.17)$$

En el contexto de la DDP, donde las consecuencias de un falso negativo son más graves que las de un falso positivo, la exactitud por sí sola no basta. Por ello se complementa con las métricas de precisión, exhaustividad y F1.

Precisión, exhaustividad y F1. La precisión mide qué proporción de las quejas clasificadas como competentes lo son realmente y penaliza las falsas alarmas:

$$\text{Precisión} = \frac{VP}{VP + FP} \quad (4.18)$$

La exhaustividad (*recall*) mide qué proporción de las quejas realmente competentes fue identificada de forma correcta y penaliza los casos omitidos:

$$\text{Exhaustividad} = \frac{VP}{VP + FN} \quad (4.19)$$

En el contexto de la DDP esta métrica tiene mayor peso institucional: una exhaustividad baja implica que quejas legítimas se están descartando sin revisión. El F1 es la media armónica entre ambas y ofrece un balance que penaliza los extremos donde una métrica es alta a costa de la otra:

$$F1 = \frac{2 \cdot \text{Precisión} \cdot \text{Exhaustividad}}{\text{Precisión} + \text{Exhaustividad}} \quad (4.20)$$

El F1 ponderado (*weighted F1*) extiende el cálculo a ambas clases, ponderando por el número de instancias de cada una:

$$F1_w = \sum_{c \in C} \frac{n_c}{N} \cdot F1_c \quad (4.21)$$

donde n_c es el número de instancias de la clase c y N el total. Esta métrica es la principal para la selección del modelo óptimo en este sistema.

Análisis de la varianza. La desviación estándar del F1 en validación cruzada (σ_{CV}) complementa el valor medio al revelar la estabilidad del modelo. Un modelo con $F1_{CV} = 0,98$ y $\sigma_{CV} = 0,04$ puede resultar menos confiable que uno con $F1_{CV} = 0,97$ y $\sigma_{CV} = 0,00$, pues el primero es sensible a la distribución particular de los datos en cada pliegue.

4.9.3. Carga perezosa y serialización con joblib

Seleccionado el modelo óptimo, este debe almacenarse y ponerse en producción de forma que garantice la reproducibilidad de sus predicciones y minimice el tiempo de arranque del sistema. Para ello se combinan la carga perezosa y la serialización con joblib, dos patrones de diseño que optimizan la eficiencia operativa del clasificador.

La carga perezosa (*lazy loading*) inicializa los recursos computacionalmente costosos —como el modelo de Sentence-BERT o el de spaCy— únicamente cuando se requieren por primera vez, y no al inicio del programa. La serialización, por su parte, convierte el objeto de Python completo —vectorizador y clasificador juntos— en un archivo binario .pkl:

```
Pipeline(vectorizador, clasificador)  $\xrightarrow{\text{joblib.dump}}$  modelo.pkl  $\xrightarrow{\text{joblib.load}}$  Pipeline lista  
para inferencia
```

Esta estrategia es técnicamente relevante por tres razones:

1. **Integridad.** Vectorizador y clasificador se serializan juntos, lo que elimina el riesgo de usar un clasificador entrenado con un vectorizador distinto al de producción.
2. **Eficiencia.** joblib emplea compresión paralela y mapeo de memoria (*memory mapping*), por lo que resulta bastante más rápido que pickle estándar para arreglos numéricos de gran tamaño, como los parámetros internos de Sentence-BERT.
3. **Reproducibilidad.** El archivo .pkl incluye la semilla aleatoria (`random_state=42`) y todos los hiperparámetros, lo que garantiza predicciones idénticas entre sesiones y entornos de ejecución.

La convención de nomenclatura de los archivos serializados es:

```
<vectorizador>__<clasificador>__<timestamp>.pkl
```

lo que permite la trazabilidad temporal de los modelos entrenados y facilita la comparación y auditoría de versiones en el contexto institucional de la DDP.

5. Análisis y Diseño

Este capítulo documenta el análisis y el diseño de la plataforma tal como quedó definida al cierre de esta etapa del Trabajo Terminal. Su propósito es dejar constancia de *qué* se construyó y, sobre todo, de *por qué* se construyó de esa manera: los requerimientos que se levantaron con la Defensoría, los actores que intervienen, los casos de uso que delimitan el alcance funcional, la arquitectura que soporta la solución y el modelo de datos que la sustenta. La construcción concreta de cada componente —el código, el despliegue y las pruebas— se documenta en el Capítulo 6, y el recorrido funcional de cada módulo entregado en el Capítulo 7.

Los artefactos que aquí se presentan siguen el orden en que fueron producidos: primero el levantamiento de requerimientos con el cliente, del que se derivaron los casos de uso; después el modelo de estados del expediente, que ordena el flujo de trabajo; sobre esa base la arquitectura de componentes y el modelo de datos; y por último las decisiones de diseño con las alternativas que se descartaron en cada punto. Este orden no es una convención editorial: cada artefacto es insumo del siguiente, y esa dependencia es la que sostiene la coherencia técnica del sistema.

5.1. Descripción general de la solución

La solución es una plataforma web para la gestión digital de quejas y denuncias ante la Defensoría de los Derechos Politécnicos. Permite a cualquier integrante de la comunidad politécnica presentar una queja y darle seguimiento —con o sin cuenta de usuario— y provee al personal de la Defensoría las herramientas para recibirla, validarla, canalizarla y administrar la operación técnica del sistema.

Al cierre de esta etapa la plataforma está compuesta por siete microservicios independientes desarrollados con Spring Boot que comparten una única base de datos PostgreSQL, tres aplicaciones web independientes desarrolladas con Angular —una por tipo de usuario— y cuatro contenedores de Nginx que actúan como proxy inverso público y como servidores de archivos estáticos. Todo el conjunto se ejecuta en contenedores sobre dos servidores virtuales privados, con acceso público cifrado mediante TLS y certificado renovado de forma automática.

5.2. Actores del sistema

El análisis identificó tres actores con necesidades, permisos e interfaces claramente diferenciadas. Esta separación es la que después justifica que existan tres aplicaciones web distintas en lugar de una sola con vistas condicionadas por rol: cada actor tiene un perfil de uso, un nivel de confianza y una superficie de exposición distintos, y mantenerlos en aplicaciones separadas reduce el riesgo de que un defecto en una interfaz exponga funcionalidad de otra.

- **Quejoso.** Cualquier integrante de la comunidad politécnica —estudiante, personal académico o administrativo— que presenta una queja o le da seguimiento. Es el único actor externo y el único que puede operar sin credenciales previas.
- **Personal de recepción.** Personal de la Defensoría encargado de recibir, validar y canalizar las quejas que llegan por el portal público o en papel.
- **Administrador de tecnologías de la información.** Responsable de la operación técnica y administrativa de la plataforma: cuentas del resto del personal, catálogo institucional, plantillas de documentos oficiales, respaldos de la base de datos y bitácora de seguridad.

El sistema define además cinco roles de personal, de los cuales solo dos cuentan con interfaz propia al cierre de esta etapa. La Tabla 5.1 resume la correspondencia entre actores, roles e interfaces.

Tabla 5.1: Actores, roles e interfaces del sistema

Actor	Rol en el sistema	Responsabilidad	Interfaz
Quejoso	— (usuario externo)	Presentar quejas y darles seguimiento	Portal público del quejoso
Personal de recepción	RECEPCIONISTA	Recibir, validar, rechazar y turnar quejas	Panel de revisión
Administrador de TI	ADMIN_SISTEMAS	Operación técnica, cuentas, catálogos, respaldos y auditoría	Consola de administración
Analista de primer contacto	ANALISTA_PRIMER_CONTACTO	Evaluación jurídica de competencia	Sin interfaz propia en esta etapa

Actor	Rol en el sistema	Responsabilidad	Interfaz
Subdefensor	SUBDEFENSOR	Solicitud de investigación y conciliación	Sin interfaz propia en esta etapa
Defensor	DEFENSOR	Resolución y cierre del expediente	Sin interfaz propia en esta etapa

Los tres roles sin interfaz propia no son un descuido del diseño: existen como cuentas capturables desde la consola de administración y el sistema los emplea como catálogo de personal disponible al momento de turnar una queja. Lo que no tienen todavía es un panel con permisos propios, y esa limitación se declara de forma explícita en el alcance de esta etapa (Sección 2.5).

5.3. Especificación de requerimientos

El levantamiento de requerimientos no se limitó a la lectura de la documentación oficial de la Defensoría, porque la operación real de una organización rara vez coincide por completo con lo que sus documentos formales describen. El equipo sostuvo una serie de reuniones con el personal de la institución que evolucionaron de forma iterativa: cada sesión aclaró aspectos que la anterior había dejado abiertos, y cada avance técnico intermedio se convirtió en insumo de la siguiente reunión.

La primera sesión tuvo como objetivo comprender el contexto operativo e identificar las áreas que intervienen en la gestión de una queja. Con ese contexto el equipo elaboró un diagrama de proceso en notación BPMN a partir de los documentos oficiales, que se presentó en una segunda sesión como base de discusión. El resultado fue el hallazgo más valioso de todo el levantamiento: el personal de la Defensoría señaló que varias etapas del procedimiento formal no se ejecutaban en la práctica, de modo que se depuraron en conjunto hasta obtener un diagrama que refleja el flujo operativo real y no el prescrito. Validado ese diagrama, se establecieron sesiones por área para traducir las actividades de cada rol en casos de uso y vistas concretas, con sesiones de revisión intermedias para confirmar que los artefactos reflejaran lo acordado.

El resultado acumulado se organizó en tres categorías —requerimientos funcionales, no funcionales y reglas de negocio— y se estructuró por actor, de modo que cada requerimiento tiene un dueño identificable. La nomenclatura empleada codifica el actor en el identificador: RF-Q para los requerimientos funcionales del quejoso, RF-R para los del personal de recepción y RF-A para los de administración, y de forma análoga RNF- y RN- para los no funcionales y las reglas de negocio. Esta convención permite rastrear cualquier elemento del diseño hasta el actor que lo motivó.

5.3.1. Requerimientos funcionales del quejoso

Los requerimientos del quejoso se agrupan en cinco bloques: operación sin cuenta, gestión de la cuenta, gestión de las quejas propias, seguimiento y servicios de apoyo. El primer bloque es el más determinante para el diseño, porque exige que el sistema acepte quejas de personas que no tienen —ni necesariamente quieren tener— una cuenta, lo que obliga a un punto de acceso público sin autenticación y a un mecanismo de identificación posterior basado en el folio.

Tabla 5.2: Requerimientos funcionales del actor Quejoso

ID	Requerimiento	Descripción
RF-Q01	Consultar una queja sin cuenta	Dado un folio y el correo con que se presentó, mostrar el detalle de la queja: motivo, descripción, estatus y fecha.
RF-Q02	Registrar una queja sin cuenta	Capturar la identidad completa del quejoso, unidad académica, fecha de los hechos, datos del denunciado, descripción libre y varios archivos de evidencia en una sola petición; generar un folio de seguimiento.
RF-Q03	Capturar datos del tutor	Si la fecha de nacimiento indica que el quejoso es menor de edad, exigir los datos de un tutor o adulto responsable antes de permitir el envío.
RF-Q04	Activar una cuenta de seguimiento	A partir de un folio y correo ya registrados en una queja existente, definir una contraseña y activar la cuenta.
RF-Q05	Iniciar sesión	Autenticarse con correo institucional y contraseña, obteniendo un token de sesión con vigencia limitada.
RF-Q06	Recuperar la contraseña	Restablecer la contraseña mediante un código de verificación de seis dígitos enviado por correo electrónico.
RF-Q07	Consultar un resumen de actividad	Mostrar al entrar al panel el conteo de quejas propias agrupadas por estatus.
RF-Q08	Listar y consultar las quejas propias	Listar las quejas del usuario con filtros por estatus y texto, y mostrar el detalle completo de cada una con su línea de tiempo de estatus.
RF-Q09	Editar una queja propia	Modificar descripción, unidad académica, fecha de hechos y datos del denunciado, únicamente mientras la queja no haya iniciado su validación.
RF-Q10	Registrar una queja autenticado	Presentar una queja nueva desde el panel sin volver a capturar la identidad, que se toma del token de sesión.
RF-Q11	Consultar las evidencias propias	Previsualizar y descargar los archivos adjuntos a una queja propia.
RF-Q12	Responder acuerdos de conciliación	Consultar los acuerdos propuestos por la Defensoría y aceptarlos o rechazarlos, con comentario opcional.
RF-Q13	Consultar el centro de notificaciones	Consultar un historial persistente de avisos con contador de no leídas y marcado individual como leída.
RF-Q14	Consultar y editar el perfil	Editar correo personal, teléfono, unidad académica y domicilio; consultar en modo de solo lectura el nombre, el correo institucional y la boleta o número de empleado.
RF-Q15	Consultar el catálogo institucional	Seleccionar la dependencia o unidad académica involucrada desde el catálogo del Instituto, en lugar de capturarla como texto libre.
RF-Q16	Consultar preguntas frecuentes	Consultar sin sesión un menú fijo de preguntas y respuestas sobre la Defensoría y el proceso de queja.
RF-Q17	Recibir avisos por correo	Recibir confirmación de creación de cuenta, confirmación de restablecimiento de contraseña y, en el caso de menores de edad, aviso al tutor.

5.3.2. Requerimientos funcionales del personal de recepción

El diseño de este bloque partió de una restricción que el propio personal planteó en las sesiones de levantamiento: no todas las quejas llegan por el portal. Una parte del volumen ingresa en papel, y el sistema debía poder incorporarlas al mismo flujo digital sin obligar al quejoso a capturarlas de nuevo. De ahí el requerimiento de registro manual (RF-R08), que es la razón por la que el modelo de datos distingue el origen de cada queja.

Tabla 5.3: Requerimientos funcionales del actor Personal de recepción

ID	Requerimiento	Descripción
RF-R01	Iniciar sesión	Autenticarse con las credenciales de personal administrativo desde el panel de revisión.
RF-R02	Consultar la bandeja de entrada	Consultar la lista de quejas por trabajar, con contadores de pendientes, en proceso y turnadas en el día.
RF-R03	Consultar el detalle de una queja	Consultar el resumen completo del expediente y los documentos adjuntos, para su validación.
RF-R04	Buscar antecedentes	Consultar otras quejas previas de la misma persona, para detectar posibles duplicados antes de turnar.
RF-R05	Consultar los documentos adjuntos	Previsualizar y descargar las evidencias asociadas a una queja.
RF-R06	Rechazar una queja	Rechazar una queja por documentación incompleta, indicando motivos y observaciones, con notificación automática al quejoso.
RF-R07	Turnar una queja	Canalizar la queja a un área o dependencia y a un defensor o subdefensor específico, con comentarios, generando el folio oficial de turnado.
RF-R08	Registrar una queja recibida en papel	Dar de alta manualmente una queja física con la identidad del quejoso, dependencia, número de oficio, fecha de recepción, tipo de documento, descripción, ubicación física del expediente y un archivo digitalizado opcional.
RF-R09	Consultar el historial de trámites	Consultar los trámites ya procesados, con filtros por texto, estatus y fecha.
RF-R10	Exportar el historial	Exportar el historial filtrado a un archivo de hoja de cálculo.
RF-R11	Emitir un acuerdo de conciliación	Emitir un acuerdo dirigido al quejoso de una queja, con asunto y términos.
RF-R12	Consultar catálogos de apoyo	Consultar la lista de dependencias y la de personal disponible para turnado.
RF-R13	Cambiar la propia contraseña	Modificar la contraseña de la cuenta propia desde el panel.

5.3.3. Requerimientos funcionales del administrador de TI

Este bloque concentra las funciones que hacen operable la plataforma a largo plazo sin intervención del equipo de desarrollo. Su requerimiento más sensible es el de respaldo y restauración (RF-A06), porque incluye la única operación destructiva de todo el sistema y por ello recibe un tratamiento de seguridad diferenciado.

Tabla 5.4: Requerimientos funcionales del actor Administrador de TI

ID	Requerimiento	Descripción
RF-A01	Iniciar sesión	Autenticarse como personal administrativo.
RF-A02	Consultar el tablero general	Consultar un resumen del estado del sistema al entrar a la consola.
RF-A03	Gestionar cuentas de personal	Listar, crear con contraseña temporal, editar, restablecer contraseña, dar de baja y reactivar cuentas de cualquiera de los cinco roles.
RF-A04	Administrar el catálogo institucional	Listar, crear y editar dependencias, e importar o actualizar el catálogo de forma masiva desde un archivo de hoja de cálculo.
RF-A05	Administrar plantillas de documentos	Listar plantillas, consultar los marcadores disponibles, previsualizar con datos de ejemplo y editar o publicar su contenido.
RF-A06	Gestionar respaldos de la base de datos	Listar los respaldos existentes, ejecutar un respaldo bajo demanda, descargar un respaldo y restaurar la base de datos a partir de uno de ellos.
RF-A07	Consultar la bitácora de acciones críticas	Consultar el registro de acciones administrativas con usuario, acción, dirección de origen y fecha.
RF-A08	Administrar el contenido de preguntas frecuentes	Listar, crear, editar y eliminar las preguntas del portal público.
RF-A09	Cambiar la propia contraseña	Modificar la contraseña de la cuenta propia desde la consola.

5.3.4. Requerimientos no funcionales

Los requerimientos no funcionales expresan atributos de calidad y no funciones. Se agruparon por la categoría del atributo que abordan —disponibilidad, confidencialidad, resiliencia, usabilidad y trazabilidad— porque esa clasificación es la que permite verificarlos: cada categoría se comprueba con un tipo distinto de prueba. La Tabla 5.5 los concentra.

Tabla 5.5: Requerimientos no funcionales de la plataforma

ID	Categoría	Atributo requerido
RNF-Q01	Disponibilidad	Debe poder presentarse una queja sin cuenta ni sesión, en cualquier momento y desde cualquier dispositivo con navegador.
RNF-Q02	Confidencialidad	Todo el tráfico público debe viajar cifrado con TLS; tras el inicio de sesión, la autenticación se transmite como token en el encabezado de autorización y nunca como credenciales en claro.
RNF-Q03	Capacidad	El proxy público debe aceptar peticiones de hasta 100 MB y el backend hasta 60 MB por petición multiparte, para permitir varios archivos de evidencia por queja.
RNF-Q04	Resiliencia	Un fallo en el envío de correo o en el registro de una notificación nunca debe impedir que la operación principal se complete; el fallo se degrada a un registro en la bitácora del servidor.

ID	Categoría	Atributo requerido
RNF-Q05	Usabilidad	Los formularios extensos deben presentarse en pasos numerados, con previsualización de los archivos seleccionados y ayuda contextual que no reduzca el espacio útil del formulario.
RNF-Q06	Seguridad	La sesión del quejoso debe expirar en una hora; al expirar o ser rechazada, el cliente debe cerrar sesión de forma automática.
RNF-Q07	Usabilidad	Toda operación debe producir retroalimentación visual inmediata de éxito, error o validación; no se admiten fallos silenciosos.
RNF-R01	Trazabilidad	Cada rechazo y cada turnado debe registrar, sin excepción, el correo de quien lo procesó y la fecha en que ocurrió.
RNF-R02	Seguridad	El personal debe autenticarse en un único punto; ningún panel de personal implementa lógica de contraseñas propia.
RNF-R03	Interoperabilidad	La exportación del historial debe producir un archivo de hoja de cálculo en formato estándar, abrible sin conversión previa.
RNF-R04	Resiliencia	Si falla el envío del correo de rechazo, el rechazo en sí debe completarse igualmente.
RNF-R05	Consistencia	Los paneles de personal deben compartir componentes visuales, patrones de error y criterios de accesibilidad.
RNF-A01	Seguridad	El primer acceso administrativo debe autoprovisionarse en el arranque inicial con una contraseña temporal registrada una sola vez en la bitácora del servidor; nunca incrustada en el cliente ni expuesta por la interfaz de programación.
RNF-A02	Trazabilidad	Toda acción administrativa relevante, no solo el inicio de sesión, debe quedar registrada con la dirección de origen.
RNF-A03	Seguridad	La restauración de la base de datos debe exigir una confirmación explícita en el cuerpo de la petición; no puede dispararse con una sola acción accidental.
RNF-A04	Tolerancia a fallos	La importación masiva del catálogo debe reportar un resumen de filas procesadas, creadas, actualizadas y con error, en lugar de fallar en bloque ante una fila mal formada.
RNF-A05	Usabilidad	La consola de administración debe aprovechar la densidad de información disponible en pantalla.
RNF-A06	Seguridad	La generación, descarga y restauración de respaldos debe estar disponible únicamente para el rol de administración de sistemas.

5.3.5. Reglas de negocio

Las reglas de negocio son las restricciones que el sistema debe hacer cumplir para que el procedimiento sea válido. A diferencia de los requerimientos funcionales, que describen lo que el sistema *permite* hacer, las reglas describen lo que *prohíbe* o *obliga*. Su característica común en esta plataforma es que se validan en el servidor y no solo en la interfaz: ocultar un botón no es hacer cumplir una regla, y así se hace explícito en varias de las que siguen.

Tabla 5.6: Reglas de negocio de la plataforma

ID	Temática	Regla
RN-Q01	Menores de edad	Una queja de una persona menor de 18 años requiere datos de tutor o adulto responsable capturados y confirmados antes de poder enviarse.
RN-Q02	Menores de edad	Una persona menor de 14 años no puede autorregistrarse ni gestionar la queja por sí misma; la interfaz indica de forma explícita que debe ser el tutor quien complete el formulario a su nombre.
RN-Q03	Identificación	El número de boleta o de empleado debe ser exclusivamente numérico y no exceder doce caracteres.
RN-Q04	Activación	Una cuenta de quejoso solo puede activarse si existe al menos una queja previa con ese folio y ese correo; el par folio-correo es la llave de activación.
RN-Q05	Activación	Una cuenta ya activa no puede volver a activarse.
RN-Q06	Contraseñas	La contraseña debe tener al menos ocho caracteres, una mayúscula y un dígito, tanto en la activación de cuenta como en el restablecimiento.
RN-Q07	Contraseñas	Un código de recuperación expira diez minutos después de solicitado y solo es válido una vez.
RN-Q08	Edición	Una queja solo es editable por su autor mientras su estatus sea RECIBIDA; en cualquier otro estatus la edición se rechaza en el servidor, no únicamente se oculta en la interfaz.
RN-Q09	Conciliación	Un acuerdo de conciliación solo puede ser respondido por el quejoso al que va dirigido; el correo del acuerdo debe coincidir con el del token de sesión.
RN-Q10	Origen	El origen de una queja se asigna de forma automática según el punto de acceso empleado —PUBLICO, AUTENTICADO o MANUAL—; no es un dato que el usuario declare.
RN-R01	Ciclo de vida	La queja transita RECIBIDA → EN_VALIDACION → RECHAZADA o TURNADA. Una vez rechazada o turnada no regresa a un estado anterior desde el panel de revisión.
RN-R02	Autorización	Únicamente las cuentas con rol de recepción pueden invocar los puntos de acceso del servicio de revisión; la restricción se declara en cada controlador y no solo a nivel de ruta.
RN-R03	Rechazo	Rechazar una queja exige al menos un motivo y genera de forma automática el correo de rechazo al quejoso; no es una acción silenciosa.
RN-R04	Turnado	Turnar una queja exige área y defensor asignado, y genera un folio oficial de turnado distinto del folio original de seguimiento.
RN-R05	Registro manual	Una queja registrada a partir de un documento físico recibe origen MANUAL y admite que no se conozca de antemano la boleta o el número de empleado.
RN-R06	Turnado	El catálogo de personal disponible para turnar incluye únicamente cuentas activas con rol de defensor o subdefensor.

ID	Temática	Regla
RN-R07	Consistencia	El servicio de quejas y el de revisión operan sobre la misma tabla física mediante entidades independientes, de modo que un cambio de estatus hecho por recepción es visible de inmediato para el quejoso sin necesidad de una llamada entre servicios.
RN-A01	Autorización	Todos los puntos de acceso administrativos exigen el rol de administración de sistemas, salvo los de autenticación y los de gestión del perfil propio, que cualquier cuenta autenticada puede usar sobre sí misma.
RN-A02	Cuentas	Una cuenta nueva de personal se crea siempre como temporal y con obligación de cambiar la contraseña en el primer inicio de sesión.
RN-A03	Cuentas	Dar de baja una cuenta la desactiva, no la elimina, para conservar la integridad referencial con las acciones que registró en el pasado.
RN-A04	Unicidad	El correo institucional y el número de empleado del personal son únicos en el sistema, garantizado por restricción de base de datos y no solo por validación de interfaz.
RN-A05	Autorización	Las rutas de administración de un catálogo deben evaluarse antes que la regla pública del mismo prefijo; sin ese orden explícito prevalecería la regla menos restrictiva por ser más genérica.
RN-A06	Plantillas	Una plantilla de documento se identifica por una clave estable de negocio y no por su identificador autogenerado, para poder referenciarla sin depender del orden de creación.
RN-A07	Trazabilidad	Cada actualización del contenido de una plantilla registra quién la actualizó y cuándo, además del asiento correspondiente en la bitácora.

5.4. Casos de uso

Los casos de uso describen las interacciones posibles entre cada actor y el sistema con independencia de la tecnología empleada. Se presentan agrupados por actor y con la estructura mínima que permite verificarlos: precondition, flujo principal y postcondición. La especificación extendida de cada caso, con flujos alternos y de excepción, se documenta en el anexo correspondiente.

Tabla 5.7: Casos de uso del actor Quejoso

ID	Nombre	Precondición	Flujo principal	Postcondición
CU-Q01	Presentar una queja sin cuenta	Ninguna	Completa el formulario público con identidad, motivo, evidencias y tutor si aplica, y lo envía	Queja creada con estatus RECIBIDA y origen PUBLICO; se muestra el folio
CU-Q02	Consultar una queja por folio	Conoce folio y correo	Captura ambos datos en la pantalla de consulta	Se muestra el detalle y el estatus de la queja

ID	Nombre	Precondición	Flujo principal	Postcondición
CU-Q03	Activar cuenta de seguimiento	Existe una queja con ese folio y correo	Define una contraseña; el sistema valida el par folio-correo contra el servicio de quejas	Cuenta activa; puede iniciar sesión
CU-Q04	Iniciar sesión	Cuenta activa	Captura correo y contraseña	Token de sesión emitido; se registra la notificación de acceso
CU-Q05	Recuperar contraseña	Cuenta activa existente	Solicita el código, lo recibe por correo y lo captura con la nueva contraseña	Contraseña actualizada; se envía correo de confirmación
CU-Q06	Presentar una queja autenticado	Sesión iniciada	Desde el panel captura motivo, descripción y evidencias	Queja creada con estatus RECIBIDA y origen AUTENTICADO
CU-Q07	Editar una queja propia	Sesión iniciada y queja en RECIBIDA	Modifica los campos habilitados desde el detalle y guarda	Queja actualizada; en otro estatus el servidor rechaza la edición
CU-Q08	Responder un acuerdo de conciliación	Existe un acuerdo pendiente dirigido a su correo	Revisa el acuerdo y lo acepta o rechaza con comentario opcional	Acuerdo pasa a aceptado o rechazado, con fecha de respuesta
CU-Q09	Revisar notificaciones	Sesión iniciada	Abre el centro de notificaciones y marca como leídas	El contador de no leídas se actualiza
CU-Q10	Actualizar el perfil	Sesión iniciada	Edita los campos habilitados y guarda	Perfil actualizado

Tabla 5.8: Casos de uso del actor Personal de recepción

ID	Nombre	Precondición	Flujo principal	Postcondición
CU-R01	Iniciar sesión en el panel de revisión	Cuenta activa con rol de recepción	Se autentica contra el servicio de administración	Token emitido; acceso al panel
CU-R02	Revisar la bandeja de entrada	Sesión iniciada	Consulta contadores y lista de pendientes	—
CU-R03	Validar y rechazar una queja	Queja en RECIBIDA o EN_VALIDACION	Revisa documentos y antecedentes, marca motivos y observaciones, confirma	Queja pasa a RECHAZADA; se envía correo al quejoso
CU-R04	Validar y turnar una queja	Queja en RECIBIDA o EN_VALIDACION	Revisa antecedentes, elige área y defensor, agrega comentarios, confirma	Queja pasa a TURNADA con folio oficial, área y defensor
CU-R05	Registrar una queja en papel	Documento físico recibido	Captura los datos del documento y adjunta el escaneo si lo hay	Queja creada con origen MANUAL y folio propio

ID	Nombre	Precondición	Flujo principal	Postcondición
CU-R06	Consultar y exportar el historial	Existen trámites procesados	Filtra por texto, estatus o fecha y exporta si lo requiere	Archivo de hoja de cálculo descargado con el filtro aplicado
CU-R07	Emitir un acuerdo de conciliación	Existe una queja con folio conocido	Captura asunto y términos dirigidos al folio y correo del quejoso	Acuerdo creado como pendiente, visible para el quejoso

Tabla 5.9: Casos de uso del actor Administrador de TI

ID	Nombre	Precondición	Flujo principal	Postcondición
CU-A01	Iniciar sesión como administrador	Cuenta activa de administración	Se autentica en la consola	Token emitido; acceso al panel
CU-A02	Dar de alta personal	Sesión iniciada	Captura nombre, número de empleado, correo y rol	Cuenta creada con contraseña temporal; queda en bitácora
CU-A03	Restablecer la contraseña de otra cuenta	La cuenta destino existe	Selecciona la cuenta y solicita el restablecimiento	Nueva contraseña temporal generada; queda en bitácora
CU-A04	Dar de baja o reactivar una cuenta	Sesión iniciada	Selecciona la cuenta y confirma la operación	La cuenta cambia su estado de actividad; queda en bitácora
CU-A05	Importar el catálogo institucional	Archivo de hoja de cálculo con el formato esperado	Carga el archivo en el módulo de catálogo	Dependencias creadas o actualizadas de forma masiva; se muestra el resumen
CU-A06	Editar una plantilla de oficio	Sesión iniciada	Edita el contenido con los marcadores disponibles, previsualiza y publica	Plantilla actualizada; queda en bitácora
CU-A07	Ejecutar un respaldo	Sesión iniciada	Solicita un respaldo bajo demanda	Archivo de respaldo generado y listado; queda en bitácora
CU-A08	Restaurar desde un respaldo	Existe al menos un respaldo	Selecciona el archivo y confirma de forma explícita la operación destructiva	Base de datos restaurada; queda en bitácora
CU-A09	Revisar la bitácora de seguridad	Sesión iniciada	Abre el historial de acciones críticas	Se listan las acciones con usuario, dirección de origen y fecha
CU-A10	Administrar las preguntas frecuentes	Sesión iniciada	Crea, edita o elimina preguntas	Los cambios se reflejan de inmediato en el portal público

5.5. Ciclo de vida del expediente

El elemento que ordena todo el flujo de trabajo es el estatus de la queja. Se modeló como una máquina de estados con cuatro valores y transiciones dirigidas, decisión que se tomó por dos

razones: permite validar en el servidor qué operaciones son admisibles en cada momento —la regla RN-Q08 depende por completo de esto— y da al quejoso una línea de tiempo comprensible del avance de su expediente.

- **RECIBIDA.** Estado inicial de toda queja, sin importar su origen. Es el único estado en el que el quejoso puede editar su propia queja, porque es el único en el que la Defensoría no ha empezado a trabajar sobre ella.
- **EN_VALIDACION.** El personal de recepción tomó la queja y está verificando su documentación y sus antecedentes. La edición por parte del quejoso queda cerrada a partir de aquí, para evitar que el expediente cambie mientras se lo evalúa.
- **RECHAZADA.** Estado terminal. La queja no reunió los requisitos de documentación; el rechazo exige motivos registrados y notificación al quejoso.
- **TURNADA.** Estado terminal desde la perspectiva de recepción. La queja fue canalizada a un área y a un defensor, y recibió un folio oficial de turnado.

Los dos estados terminales lo son únicamente respecto del alcance implementado en esta etapa: el flujo completo de la Defensoría continúa después del turnado con la investigación, la conciliación y la resolución, etapas que corresponden a los roles que aún no cuentan con interfaz propia. El modelo de datos ya contempla los campos necesarios para esas etapas, de modo que su incorporación no requerirá modificar el esquema existente.

5.6. Diseño de la arquitectura

La arquitectura de la plataforma aplica los principios expuestos en la Sección 4.1. Esta sección describe cómo se concretaron: qué componentes existen, cómo se comunican, cómo se despliegan y qué alternativas se descartaron en cada decisión.

5.6.1. Vista de componentes

El sistema se descompuso en siete microservicios, cada uno con una responsabilidad acotada y su propio ciclo de despliegue. El criterio de división no fue técnico sino de dominio: cada servicio agrupa las operaciones que comparten un mismo conjunto de datos y una misma razón para cambiar. La Tabla 5.10 presenta la descomposición resultante.

Tabla 5.10: Microservicios de la plataforma y su responsabilidad

Servicio	Puerto	Responsabilidad	Emite tokens
auth-service	8083	Sesión del quejoso, activación de cuenta, recuperación de contraseña y perfil	Sí, para quejosos
queja-service	8084	Registro de quejas público y autenticado, consulta y edición de quejas propias, evidencias y respuesta a acuerdos de conciliación	No, solo verifica
notificaciones-service	8085	Envío de correo electrónico y centro de notificaciones persistido por usuario	No, solo verifica

Servicio	Puerto	Responsabilidad	Emite tokens
catalogo-service	8086	Catálogo de dependencias y unidades académicas del Instituto	No, solo verifica
admin-service	8087	Sesión del personal, cuentas y roles, plantillas de oficios, respaldos y bitácora de seguridad	Sí, para personal
revision-service	8088	Bandeja de recepción, validación, rechazo, turnado, registro manual, historial exportable y emisión de acuerdos	No, solo verifica
chatbot-service	8089	Menú de preguntas frecuentes del portal público y su administración	No, solo verifica

Solo dos de los siete servicios emiten tokens de sesión: uno para quejosos y otro para personal. Los cinco restantes se limitan a verificarlos. Esta asimetría es deliberada y responde al principio de un único punto de emisión de credenciales: concentrar la lógica de contraseñas en dos lugares en vez de siete reduce la superficie donde puede introducirse un defecto de autenticación, y evita que la política de complejidad de contraseñas quede implementada de siete maneras distintas.

5.6.2. Vista de despliegue

El sistema se despliega sobre dos servidores virtuales privados con responsabilidades separadas. El primero, al que en adelante se denomina *servidor de aplicación*, aloja los siete microservicios y la base de datos. El segundo, el *servidor de acceso público*, aloja las tres aplicaciones web y el proxy inverso que da la cara a Internet.

Esta separación cumple tres propósitos. El primero es de seguridad: el servidor que contiene la base de datos y la lógica de negocio no está expuesto directamente a Internet, y su cortafuegos acepta conexiones a los puertos de los microservicios únicamente desde la dirección del servidor de acceso público. El segundo es de aislamiento de fallos: una saturación en la entrega de archivos estáticos no compite por recursos con el procesamiento de las peticiones de negocio. El tercero es operativo: el certificado de transporte y la configuración del dominio residen en un solo lugar, de modo que renovarlos no implica reiniciar ningún servicio de negocio.

El único puerto realmente público es el de HTTPS; el puerto de HTTP se conserva abierto exclusivamente para redirigir el tráfico a su equivalente cifrado y para atender el reto de validación del certificado. La Tabla 5.11 presenta las reglas de enrutamiento del proxy inverso, que son el contrato entre el exterior y la arquitectura interna.

Tabla 5.11: Reglas de enrutamiento del proxy inverso público

Ruta pública	Destino	Servidor
/	Portal del quejoso	Acceso público
/admin/	Consola de administración	Acceso público
/revision/	Panel de revisión	Acceso público
/api/auth/	auth-service	Aplicación
/api/quejoso/	queja-service	Aplicación
/api/notificaciones/	notificaciones-service	Aplicación
/api/catalogos/	catalogo-service	Aplicación
/api/admin/	admin-service	Aplicación

Ruta pública	Destino	Servidor
/api/revision/	revision-service	Aplicación
/api/chatbot/	chatbot-service	Aplicación

Una consecuencia de diseño de este esquema merece destacarse: como el proxy entrega tanto las aplicaciones web como la interfaz de programación bajo un mismo dominio, el navegador considera que todo proviene del mismo origen y no activa la verificación de intercambio de recursos entre orígenes. Eso simplifica el modelo de seguridad del cliente sin sacrificar la separación en servicios del lado del servidor.

5.6.3. Diseño de la comunicación entre servicios

La comunicación entre servicios es síncrona sobre HTTP siguiendo el estilo REST, conforme a lo expuesto en la Sección 4.1. Se descartó la mensajería asíncrona porque el volumen de operaciones y el número de servicios no justifican la complejidad operativa de un intermediario de mensajes, y porque ninguno de los flujos identificados tolera bien la consistencia eventual: un quejoso que envía una queja espera su folio en la misma respuesta, no después.

El diseño reduce al mínimo las llamadas entre servicios. Solo existen dos rutas de comunicación interna: la validación del par folio-correo que el servicio de autenticación solicita al de quejas durante la activación de una cuenta, y el envío de correo que varios servicios solicitan al de notificaciones. Todo lo demás se resuelve por lectura directa de la base de datos compartida, decisión cuyas consecuencias se discuten en la Sección 5.6.5.

5.6.4. Diseño de la seguridad y el control de acceso

El esquema de seguridad se construyó sobre cuatro mecanismos, cada uno atendiendo una amenaza distinta.

La **autenticación** emplea tokens firmados con algoritmo simétrico. Un token emitido por el servicio de autenticación es válido en los servicios que atienden al quejoso, y uno emitido por el servicio de administración es válido en los que atienden al personal, porque todos comparten la misma clave de firma. Cada servicio verifica el token de forma local, sin consultar al emisor, lo que evita un punto único de falla en cada petición. La vigencia del token es de una hora, y el cliente cierra la sesión de forma automática cuando el servidor la rechaza.

El **almacenamiento de contraseñas** usa una función de resumen con costo configurable en los dos servicios que emiten sesión. La política de complejidad exige un mínimo de ocho caracteres, al menos una mayúscula y al menos un dígito, y está implementada como una expresión compartida para que no divergja entre servicios. Las cuentas de personal se crean con contraseña temporal y obligación de cambiarla en el primer acceso.

La **autorización** se declara por rol en cada controlador, no solo por ruta. Esta distinción no es cosmética: una restricción declarada únicamente en la configuración de rutas puede eludirse si una ruta nueva se agrega sin actualizar esa configuración, mientras que la declaración en el controlador viaja con el código que protege.

El **cifrado en tránsito** se termina en el proxy inverso con un certificado de una autoridad reconocida y renovación automática. Del proxy hacia los microservicios el tráfico viaja sin cifrar, decisión aceptable porque ese tramo no atraviesa Internet y está restringido por dirección de origen en el cortafuegos, pero que conviene revisar si en el futuro los servidores dejan de estar bajo control directo del proyecto.

Un punto pendiente que debe declararse: el cierre automático de sesión por inactividad solicitado para las cuentas de personal no está implementado al cierre de esta etapa. La expiración del token limita la ventana de exposición a una hora, pero no equivale a un cierre por inactividad.

5.6.5. Decisiones de diseño y alternativas descartadas

Esta sección documenta las decisiones estructurales del proyecto con el formato contexto-alternativas-decisión-consecuencias. Registrarlas de esta forma cumple un propósito concreto: una decisión de arquitectura cuyo razonamiento no se documenta se vuelve indistinguible de un accidente, y quien mantenga el sistema en el futuro no sabrá si puede revertirla.

Decisión 1: base de datos única compartida. *Contexto:* siete servicios independientes necesitan datos que se relacionan entre sí —una queja, su tutor, sus evidencias, el usuario que la presentó, la notificación que se le envió—. *Alternativas:* una base de datos por servicio, con sincronización explícita entre ellas; o una base única compartida. *Decisión:* base única. *Consecuencias aceptadas:* se sacrifica la autonomía de datos que caracteriza a los microservicios puros, y dos servicios escriben la misma tabla sin coordinación transaccional entre ellos. A cambio se evita la complejidad de mantener consistencia entre bases separadas, que a la escala de este sistema habría requerido un esfuerzo mayor que el beneficio obtenido. Es un híbrido consciente, no una implementación incompleta del patrón.

Decisión 2: entidades independientes sobre una tabla compartida. *Contexto:* el servicio de quejas y el de revisión necesitan la misma tabla de quejas, pero con vistas distintas: el primero requiere los campos del quejoso, el segundo los del flujo de validación. *Alternativas:* un servicio dueño de la tabla que exponga los datos al otro por interfaz de programación; o dos clases de persistencia independientes sobre la misma tabla física. *Decisión:* entidades independientes. *Consecuencias aceptadas:* un cambio de estatus hecho por recepción es visible de inmediato para el quejoso sin llamada entre servicios, lo que simplifica el flujo; pero ambos servicios pueden agregar columnas sin que el otro lo sepa, y una modificación descoordinada del esquema puede romper al servicio vecino. La mitigación adoptada es que cada servicio declare únicamente las columnas que usa.

Decisión 3: catálogo institucional como servicio propio. *Contexto:* el catálogo de dependencias del Instituto es requerido por el formulario público de quejas, por el turnado en el panel de revisión y por la consola de administración. *Alternativas:* alojarlo dentro del servicio de quejas, que fue quien primero lo necesitó; o extraerlo a un servicio propio. *Decisión:* servicio propio. *Consecuencias aceptadas:* se añadió un servicio más que operar y desplegar, con el sobre costo correspondiente, en un momento en que un solo consumidor lo necesitaba. La razón para asumirlo fue anticipada: al convertirse en dato de referencia de tres consumidores distintos, alojarlo dentro de un servicio de negocio habría obligado a que los otros dos dependieran de él por razones ajenas a su dominio.

Decisión 4: evidencias almacenadas en la base de datos. *Contexto:* cada queja puede llevar varios archivos de evidencia que constituyen material probatorio. *Alternativas:* guardar los archivos en el sistema de archivos del servidor y almacenar la ruta en la base; o almacenar el contenido binario completo dentro de la base. *Decisión:* contenido binario en la base. *Consecuencias aceptadas:* la base crece más rápido y los respaldos son más grandes. A cambio se

obtiene lo que en este dominio pesa más: el archivo y su expediente se respaldan y se restauran como una unidad atómica, de modo que no puede existir un expediente cuya evidencia se haya perdido porque el respaldo del sistema de archivos y el de la base se tomaron en momentos distintos. Para material con posible valor probatorio, esa garantía justifica el costo de almacenamiento.

Decisión 5: tres aplicaciones web independientes. *Contexto:* tres actores con perfiles de uso y niveles de privilegio muy distintos. *Alternativas:* una sola aplicación con vistas y rutas condicionadas por rol; o tres aplicaciones separadas. *Decisión:* tres aplicaciones. *Consecuencias aceptadas:* se duplica cierto código de infraestructura —guardas, interceptores, estilos base— en tres proyectos que deben mantenerse sincronizados a mano. A cambio, el código de la consola de administración no se entrega al navegador de un usuario público, lo que reduce la superficie expuesta: un visitante del portal no recibe siquiera las rutas de administración.

Decisión 6: datos del quejoso en columnas propias. *Contexto:* el registro público debe capturar la identidad completa del quejoso, que en una versión inicial se concatenaba como texto libre dentro de la descripción de la queja. *Alternativas:* conservar la concatenación, más simple de implementar; o estructurar cada dato en su propia columna. *Decisión:* columnas propias. *Consecuencias aceptadas:* el esquema creció y el formulario requiere más validación. A cambio, los datos se volvieron consultables y verificables —la búsqueda de antecedentes del requerimiento RF-R04 sería impracticable sobre texto concatenado— y la unidad académica pudo pasar de texto libre a una referencia al catálogo institucional.

Decisión 7: una imagen de contenedor por servicio. *Contexto:* los microservicios comparten la misma imagen base y el mismo procedimiento de construcción. *Alternativas:* una imagen común reutilizada por todos, parametrizada al ejecutarse; o una imagen dedicada por servicio. *Decisión:* una por servicio. *Consecuencias aceptadas:* se consume más espacio de almacenamiento en el registro de imágenes. A cambio, cada contenedor es identificable y versionable de forma independiente, y se elimina la ambigüedad de no poder saber qué versión de qué servicio está ejecutando un contenedor determinado.

5.7. Diseño de la base de datos

La base de datos es relacional y única para los siete microservicios. Cada tabla tiene un servicio propietario, entendido como el único autorizado a definir su estructura, y en dos casos una segunda tabla es escrita por un servicio adicional bajo el patrón descrito en la Decisión 2. La Tabla 5.12 presenta el esquema con su propietario y su contenido.

Tabla 5.12: Esquema de la base de datos por tabla y servicio propietario

Tabla	Propietario	Contenido
usuarios	auth-service	Cuentas de quejoso: identidad institucional, credencial cifrada, datos de contacto, datos de tutor y código de recuperación con su vigencia

Tabla	Propietario	Contenido
quejas	queja-service y revision-service	Cuerpo del expediente: folio, identidad del quejoso, narrativa de los hechos, unidad académica, denunciado, origen, estatus, y los campos del flujo de validación —motivo de rechazo, área, defensor asignado, quién validó y cuándo—
queja_tutores	queja-service	Datos del tutor o adulto responsable, en relación uno a uno con la queja
queja_evidencias	queja-service	Archivos de evidencia: nombre, tipo, tamaño y contenido binario completo
acuerdos_conciliacion	revision-service y queja-service	Acuerdos emitidos por la Defensoría: asunto, términos, estado y respuesta del quejoso
notificaciones	notificaciones-service	Avisos persistidos por destinatario, con tipo, estado de lectura y enlace
dependencias	catalogo-service	Catálogo institucional jerárquico con clave propia, clave del padre, tipo, categoría y nivel
personal_administrativo	admin-service	Cuentas de personal: identidad, rol, credencial cifrada, condición de cuenta temporal y último acceso
plantillas_documentos	admin-service	Plantillas de oficios con clave estable de negocio, contenido y datos de última actualización
bitacora_acciones	admin-service	Registro de auditoría: usuario, acción, dirección de origen y fecha
preguntas_chatbot	chatbot-service	Contenido del menú de preguntas frecuentes del portal público

5.7.1. Correspondencia entre reglas de negocio y mecanismos de integridad

Un modelo de datos que solo almacena información deja las reglas del dominio a merced de que cada consumidor las respete. El diseño de esta base busca lo contrario: que las reglas se hagan cumplir en el nivel más bajo posible, de modo que ningún camino de acceso pueda violarlas. La Tabla 5.13 muestra esa correspondencia para las reglas que admiten verificación estructural.

Tabla 5.13: Reglas de negocio y mecanismo que las hace cumplir		
Regla	Enunciado	Mecanismo
RN-Q04	El par folio-correo es la llave de activación	Consulta entre servicios que exige coincidencia de ambos campos antes de crear la cuenta
RN-Q05	Una cuenta activa no se reactiva	Verificación del indicador de actividad antes de procesar la activación
RN-Q07	El código de recuperación expira en diez minutos	Columna de fecha de expiración evaluada en el servidor al validar el código
RN-Q08	Una queja solo es editable en estatus recibida	Validación del estatus en la capa de negocio antes de aplicar la modificación
RN-Q09	Solo el destinatario responde un acuerdo	Comparación del correo del acuerdo contra el del token de sesión

Regla	Enunciado	Mecanismo
RN-Q10	El origen de la queja se infiere	Asignación en el servidor según el punto de acceso invocado, sin exponerlo como campo de entrada
RN-R01	Transiciones válidas del expediente	Columna de estatus con conjunto acotado de valores y validación de la transición en la capa de negocio
RN-R02	Solo recepción usa el servicio de revisión	Declaración de rol requerido en cada controlador
RN-R06	Solo defensores activos reciben turnado	Consulta filtrada por rol y por indicador de actividad
RN-A03	La baja desactiva, no elimina	Indicador de actividad en lugar de borrado físico, preservando la integridad referencial con la bitácora
RN-A04	Correo y número de empleado únicos	Restricción de unicidad en la definición de la tabla
RN-A06	La plantilla se identifica por clave de negocio	Restricción de unicidad sobre la columna de tipo, independiente del identificador autogenerado

El diccionario de datos completo, con el tipo y la longitud de cada columna, las cardinalidades detalladas, los índices y las sentencias de creación, se documenta en el anexo correspondiente.

5.8. Diseño del clasificador de competencia

El clasificador de competencia se diseñó como componente independiente de la plataforma, con la función de sugerir al personal de primer contacto si una queja es competencia de la Defensoría a partir de la narrativa de los hechos. Sus fundamentos conceptuales y su diseño detallado —la representación híbrida de características, el factor de amplificación de la señal normativa, la arquitectura del flujo de transformaciones y el protocolo de evaluación— se desarrollan en las secciones 4.4 a 4.9 del marco teórico, por lo que aquí se documenta únicamente su relación con el resto del sistema.

El diseño de la integración prevé que el clasificador se consuma como un servicio adicional invocado por el flujo de validación, de modo que su ausencia no interrumpa la operación administrativa. Esta condición se recogió como requerimiento explícito: la indisponibilidad del componente inteligente no debe impedir que una queja se reciba, se valide o se turne. La consecuencia de diseño es que la sugerencia del clasificador es un dato adicional del expediente, nunca una precondition de su avance, lo cual además satisface la restricción de que la decisión final sea siempre humana.

Al cierre de esta etapa el clasificador se desarrolló y evaluó de forma independiente —los resultados experimentales se presentan en el Capítulo 6— y no forma parte de los siete microservicios desplegados. Su integración como servicio de la plataforma está prevista para la etapa siguiente.

5.9. Diseño de la interfaz de usuario

El diseño de las interfaces partió de prototipos de alta fidelidad que se usaron como herramienta de validación temprana con el cliente. La razón de invertir en ellos antes de escribir código

fue económica: corregir el orden de los campos de un formulario en un prototipo cuesta minutos, y hacerlo sobre una interfaz ya implementada y desplegada cuesta un ciclo completo de compilación y despliegue.

Esa inversión se justificó en la práctica. La retroalimentación de las sesiones de revisión derivó en dos tipos de cambio que se incorporaron antes de implementar: ajustes en el orden y agrupación de los campos del formulario de queja, y la incorporación de mensajes de validación explícitos donde el prototipo original solo mostraba el campo en rojo.

Los criterios de diseño que se fijaron para las tres aplicaciones fueron los siguientes:

- **Identidad institucional.** La paleta y la estructura del encabezado y del pie siguen la identidad visual del Instituto, de modo que el usuario reconozca que se encuentra en un servicio oficial y no en un sitio de terceros.
- **Formularios por pasos.** Los formularios extensos se dividen en secciones numeradas en lugar de presentarse como una lista continua de campos, para reducir la carga cognitiva y hacer visible el progreso.
- **Ayuda contextual no intrusiva.** Los avisos normativos que el usuario debe conocer — como el plazo para presentar una queja— se presentan en un elemento flotante consultable en cualquier momento, en lugar de un bloque fijo que reduce el espacio útil del formulario.
- **Retroalimentación explícita.** Toda operación produce un mensaje visible de éxito, error o validación pendiente. Este criterio se elevó a requerimiento no funcional (RNF-Q07) porque su ausencia fue la causa de que, en una versión temprana, los mensajes de error existieran en el código pero no fueran visibles para el usuario.
- **Consistencia entre paneles de personal.** La consola de administración y el panel de revisión comparten componentes, patrones de error y estilos base, para que el personal que use ambos no tenga que aprender dos lenguajes visuales distintos.

Los prototipos completos y las capturas de las interfaces implementadas se presentan en el Capítulo 7 y en el anexo correspondiente.

6. Implementación y evaluación del prototipo

Este capítulo documenta cómo se construyó, se desplegó y se verificó la plataforma diseñada en el Capítulo 5.1. Se organiza siguiendo las capas del sistema: primero el entorno y las herramientas, después la implementación del lado del servidor, en seguida las aplicaciones web, luego el empaquetado en contenedores y la publicación en los servidores, y finalmente la evaluación del clasificador y las pruebas realizadas. Cierra con un balance explícito de lo que quedó implementado y lo que no, porque un documento técnico que solo enumera logros no permite a nadie retomar el trabajo.

6.1. Entorno de desarrollo y herramientas

La plataforma se construyó con el conjunto de tecnologías cuyos fundamentos se expusieron en la Sección 4.2. La Tabla 6.1 resume la asignación concreta de cada herramienta a su función dentro del proyecto.

Tabla 6.1: Herramientas empleadas y su función en el proyecto

Capa	Herramienta	Función
Lenguaje del servidor	Java 21	Lenguaje de los siete microservicios
Marco del servidor	Spring Boot	Autoconfiguración, servidor embebido, seguridad y persistencia
Construcción	Maven	Gestión de dependencias y empaquetado en archivo ejecutable
Persistencia	Spring Data JPA	Mapeo objeto-relacional y derivación de consultas
Base de datos	PostgreSQL 16	Almacén relacional único compartido
Autenticación	JSON Web Token	Tokens firmados con algoritmo simétrico
Documentación de la API	OpenAPI 3	Especificación e interfaz de exploración por servicio
Hojas de cálculo	Apache POI	Generación del historial exportable y lectura de la importación masiva del catálogo
Cliente web	Angular 21	Las tres aplicaciones de página única
Contenedores	Podman	Empaquetado y ejecución de los doce contenedores
Servidor web	Nginx	Proxy inverso público y entrega de archivos estáticos
Certificados	Certbot	Emisión y renovación automática del certificado de transporte
Control de versiones	Git	Historial del proyecto
Clasificador	Python, scikit-learn, spaCy, Sentence-BERT	Vectorización, entrenamiento y evaluación del componente de clasificación

Una característica del proyecto que conviene señalar desde el principio: cada microservicio es un proyecto de construcción independiente, sin un descriptor padre que los agrupe. La decisión simplifica la evolución de cada servicio por separado, pero tiene una consecuencia operativa concreta: no existe un comando único que compile todo el sistema, y cada servicio debe compilarse desde su propio directorio.

6.2. Implementación del lado del servidor

Los siete microservicios comparten deliberadamente la misma estructura interna. Esta uniformidad fue una decisión de implementación, no una consecuencia: cuando se construyó cada servicio nuevo se replicó la organización de carpetas, los filtros y el manejo de errores del anterior, en lugar de introducir un estilo distinto. El beneficio es que quien conoce un servicio conoce los siete, y un defecto corregido en uno se localiza sin dificultad en los demás.

6.2.1. Estructura común de un microservicio

Cada servicio organiza su código en cinco paquetes que corresponden a las capas descritas en la Sección 4.1:

- **controller**. Puntos de acceso HTTP. Reciben y devuelven objetos de transferencia, nunca entidades de persistencia, y declaran el rol requerido para invocarlos.
- **service**. Lógica de negocio y validación de las reglas. Es la única capa que conoce las reglas del dominio; los controladores no validan reglas y los repositorios no las conocen.
- **repository**. Acceso a datos mediante interfaces de persistencia que derivan las consultas a partir del nombre de sus métodos, sin escribir sentencias SQL.
- **entity**. Entidades de persistencia con su correspondencia a tablas y columnas.
- **model**. Objetos de transferencia de datos para las peticiones y las respuestas.

La separación entre `entity` y `model` merece una justificación explícita, porque implica escribir clases que a primera vista parecen redundantes. Exponer directamente la entidad de persistencia en una respuesta HTTP acopla el contrato público del servicio a la estructura de la base de datos: cualquier columna nueva aparece automáticamente en la respuesta, incluidas las que no deberían ser públicas —la credencial cifrada de un usuario, por ejemplo—. Mantener objetos de transferencia separados obliga a decidir de forma consciente qué se expone.

6.2.2. Verificación de tokens y control de acceso

El mecanismo de autenticación se implementó como un filtro que se ejecuta antes de que la petición llegue al controlador. Su lógica es la misma en los siete servicios: extrae el token del encabezado de autorización, verifica su firma con la clave compartida, extrae de él la identidad y el rol, y establece el contexto de seguridad de la petición. Si el token falta, está vencido o su firma no coincide, el filtro rechaza la petición sin llegar a la lógica de negocio.

Sobre ese contexto se aplica la autorización, declarada en cada controlador mediante anotaciones que exigen un rol determinado. Un detalle de configuración que resultó relevante durante la implementación: cuando un mismo prefijo de ruta tiene una parte pública y una parte administrativa, el orden de declaración de las reglas importa, porque el marco de seguridad aplica la primera coincidencia. Declarar la regla pública antes de la administrativa hace que la menos restrictiva prevalezca por ser más genérica, y el resultado es un punto de acceso administrativo efectivamente abierto. Esta situación se atendió declarando siempre la regla más específica primero, y quedó registrada como regla de negocio RN-A05 justamente para que no se reintroduzca al agregar servicios nuevos.

6.2.3. Manejo centralizado de errores

Cada servicio implementa un manejador global de excepciones que traduce cualquier error de la capa de negocio a una respuesta con estructura uniforme: mensaje, marca de tiempo y código. La alternativa —capturar excepciones en cada controlador— se descartó porque produce respuestas de error con formatos distintos según el punto de acceso, y obliga al cliente a implementar tantas variantes de manejo de error como puntos de acceso consume. Con el manejador centralizado, las tres aplicaciones web leen el mensaje de error siempre del mismo lugar.

6.2.4. Documentación de la interfaz de programación

Los siete servicios exponen su especificación en formato OpenAPI y una interfaz de exploración interactiva. Esta decisión resolvió un problema concreto de coordinación: durante el desarrollo

del cliente, la única forma de conocer la forma exacta de una petición era leer el código del controlador correspondiente. Con la especificación publicada, el contrato de cada servicio es consultable sin abrir el código.

6.2.5. Puntos de acceso implementados

La Tabla 6.2 presenta los puntos de acceso principales agrupados por servicio, con su condición de acceso. La especificación completa de cada uno, con sus parámetros y respuestas, se consulta en la interfaz de exploración de cada servicio y se reproduce en el anexo correspondiente.

Tabla 6.2: Puntos de acceso principales por microservicio

Servicio	Punto de acceso	Acceso
auth-service	POST /api/auth/login	Público
	POST /api/auth/activar-cuenta	Público
	POST /api/auth/solicitar-codigo	Público
	POST /api/auth/reset-password	Público
	GET y PUT /api/auth/me, /api/auth/perfil	Sesión de quejoso
queja-service	POST /api/quejoso/quejas/registro-publico	Público
	GET /api/quejoso/quejas/folio/{folio}	Público
	POST /api/quejoso/quejas/registrar	Sesión de quejoso
	GET /api/quejoso/quejas/mias	Sesión de quejoso
	GET y PUT /api/quejoso/quejas/mias/{folio}	Sesión de quejoso
notificaciones-service	GET y PUT /api/quejoso/conciliaciones/**	Sesión de quejoso
	GET /api/notificaciones/mias y /mias/no-leidas	Sesión de quejoso
	PUT /api/notificaciones/{id}/leida	Sesión de quejoso
	POST /api/notificaciones/enviar y /registrar	Uso interno entre servicios
	GET /api/catalogos/dependencias y /dependencias/{clave}	Público
catalogo-service	GET, POST, PUT /api/catalogos/dependencias/admin/**	Administración
	POST /api/catalogos/dependencias/admin/importar-excel	Administración
admin-service	POST /api/admin/auth/login	Público
	GET, POST, PUT, DELETE /api/admin/personal/**	Administración
	GET y PUT /api/admin/plantillas/**	Administración
	GET y POST /api/admin/seguridad/respaldos/**	Administración
	POST /api/admin/seguridad/restaurar	Administración
	GET /api/admin/seguridad/bitacora	Administración
	GET /api/admin/dashboard/resumen	Administración
	PUT /api/admin/perfil/password	Cualquier cuenta de personal
revision-service	GET /api/revision/bandeja	Recepción
	GET /api/revision/quejas/{folio} y /antecedentes	Recepción
	GET /api/revision/quejas/evidencias/{id}	Recepción
	POST /api/revision/quejas/{folio}/rechazar y /turnar	Recepción

Servicio	Punto de acceso	Acceso
	POST /api/revision/registro-manual	Recepción
	GET /api/revision/historial y /historial/exportar	Recepción
	GET /api/revision/catalogos/areas y /defensores	Recepción
	POST y GET /api/revision/conciliaciones	Recepción
chatbot-service	GET /api/chatbot/menu	Público
	GET, POST, PUT, DELETE /api/chatbot/admin/preguntas/**	Administración

6.3. Implementación de las aplicaciones web

Las tres aplicaciones se desarrollaron como aplicaciones de página única con componentes autónomos, sin el sistema de módulos de versiones anteriores del marco de trabajo. Cada aplicación es un proyecto independiente con su propio árbol de rutas, su guarda de autenticación y sus interceptores.

6.3.1. Estructura común

Las tres comparten la misma organización: una capa core con los servicios de comunicación con el servidor, los modelos de datos y la configuración de seguridad del cliente; una capa shared con los componentes reutilizables —encabezado, pie, avisos, ayuda flotante—; y una capa pages con un componente por pantalla, cargado de forma diferida para que el navegador no descargue el código de pantallas que el usuario no ha visitado.

6.3.2. Interceptores y guardas

Dos piezas transversales resuelven problemas que de otro modo habría que repetir en cada pantalla.

El **interceptor de autorización** agrega el token de sesión al encabezado de cada petición saliente, de modo que ningún servicio de la aplicación tiene que recordar hacerlo. El mismo interceptor detecta las respuestas de sesión inválida o insuficiente y, en ese caso, cierra la sesión local y redirige al inicio de sesión. Esto implementa el requerimiento RNF-Q06 en un solo punto del código.

La **guarda de rutas** impide el acceso a las secciones autenticadas cuando no existe una sesión válida en el cliente. Es importante entender su alcance: la guarda es una medida de usabilidad, no de seguridad, porque se ejecuta en el navegador y puede eludirse. La seguridad real la aplica el servidor, que rechaza cualquier petición sin token válido con independencia de lo que haga el cliente. Confundir ambas cosas es un error frecuente y por eso se hace explícito.

6.3.3. Detección de cambios sin zonas

La versión del marco de trabajo empleada permite ejecutar la aplicación sin la biblioteca que tradicionalmente intercepta las operaciones asíncronas para disparar la actualización de la vista. Ese modo reduce el trabajo del navegador, pero tiene una consecuencia que apareció durante la implementación: la vista no se actualizaba de forma automática al recibir la respuesta de una petición, de modo que los datos llegaban pero la pantalla seguía mostrando el estado anterior.

La solución adoptada fue un interceptor adicional que solicita de forma explícita una actualización de la vista tras cada respuesta del servidor. Se documenta aquí porque es una particularidad no evidente del entorno y porque cualquier pantalla nueva que se agregue al proyecto depende de que ese interceptor siga registrado.

6.4. Empaquetado en contenedores y despliegue

El sistema se ejecuta en doce contenedores distribuidos en los dos servidores descritos en la Sección 5.6. La Tabla 6.3 presenta la matriz de despliegue.

Tabla 6.3: Matriz de despliegue en contenedores

Contenedor	Servidor	Contenido
Base de datos	Aplicación	PostgreSQL 16 con almacenamiento persistente
auth-service	Aplicación	Imagen dedicada del servicio de autenticación
quejas-service	Aplicación	Imagen dedicada del servicio de quejas
notificaciones-service	Aplicación	Imagen dedicada del servicio de notificaciones
catalogo-service	Aplicación	Imagen dedicada del servicio de catálogo
admin-service	Aplicación	Imagen propia que incluye el cliente de la base de datos, necesario para generar y restaurar respaldos
revision-service	Aplicación	Imagen dedicada del servicio de revisión
chatbot-service	Aplicación	Imagen dedicada del servicio de preguntas frecuentes
Portal del quejoso	Acceso público	Servidor de estáticos con la aplicación compilada
Consola de administración	Acceso público	Servidor de estáticos con la aplicación compilada
Panel de revisión	Acceso público	Servidor de estáticos con la aplicación compilada
Proxy inverso	Acceso público	Nginx con la configuración de enrutamiento y el certificado de transporte

6.4.1. Configuración externalizada

Cada microservicio monta su configuración de producción como un volumen del contenedor, en lugar de incluirla en la imagen. La consecuencia práctica es una distinción importante en el procedimiento de operación: un cambio de configuración —una dirección, un tiempo de expiración— se aplica editando el archivo montado y reiniciando el contenedor, mientras que un cambio de lógica exige recompilar el servicio, generar un artefacto nuevo y reconstruir la imagen. Esta separación es la que permite que la misma imagen sirva en cualquier ambiente sin reconstruirse, tal como se expuso en la Sección 4.2.

Durante la operación apareció una consecuencia no evidente de este esquema: editar el archivo de configuración en el servidor anfitrión no basta para que el contenedor lo tome, porque el punto de montaje conserva la referencia al archivo original. Es necesario reiniciar el contenedor, no solo recargar el servicio. Se documenta porque es una fuente de diagnósticos erróneos: el archivo se ve correcto en el servidor y el servicio sigue comportándose como antes.

6.4.2. Publicación segura y cifrado del canal

La publicación al exterior se resolvió en tres pasos. Primero se restringió el cortafuegos del servidor de aplicación para que los puertos de los microservicios y de la base de datos aceptaran conexiones únicamente desde la dirección del servidor de acceso público, dejando fuera cualquier otro origen de Internet. Después se configuró el proxy inverso del servidor de acceso público con las reglas de enrutamiento de la Tabla 5.11. Por último se emitió el certificado de transporte mediante el método de validación por directorio web, se reconfiguró el proxy para atender el puerto cifrado y se dejó el puerto sin cifrar únicamente para redirigir el tráfico y atender la renovación automática del certificado.

Durante este proceso se documentaron dos incidentes que conviene registrar por su valor diagnóstico. El primero fue una aparente imposibilidad de alcanzar los puertos web del servidor de acceso público desde el exterior, mientras que el puerto de administración remota respondía con normalidad. Tras descartar de forma sistemática la configuración del servicio, la publicación del contenedor y el cortafuegos local, se identificó que la causa era una regla de cortafuegos del proveedor que estaba guardada pero no sincronizada con la máquina virtual: existía en el panel de control pero no se había aplicado. El segundo fue un aviso recurrente del motor de contenedores sobre una red virtual mal configurada, que provenía de un intento previo de crear una red propia que finalmente no se utilizó; su eliminación resolvió el aviso. Ambos casos ilustran un principio de diagnóstico útil: cuando un componente responde de forma local y no desde el exterior, el problema no está en el componente sino en el camino.

6.5. Evaluación del clasificador de competencia

El clasificador se evaluó de forma independiente, conforme al protocolo definido en la Sección 4.9. Se construyeron ocho configuraciones que combinan las dos representaciones semánticas del texto con los cuatro clasificadores descritos en el marco teórico, y se evaluaron todas bajo el mismo procedimiento para que la comparación fuera válida.

El protocolo aplicado tiene tres características que conviene explicitar, porque son las que impiden que las métricas resulten optimistas. La primera es la partición en tres subconjuntos: entrenamiento, validación cruzada dentro del entrenamiento, y un subconjunto de prueba aislado por completo durante el ajuste de hiperparámetros. La segunda es la estratificación de los pliegues, que preserva la proporción de clases en cada uno y evita que un pliegue quede sin ejemplos de la clase minoritaria. La tercera, y la más determinante, es que el criterio de selección del modelo final es su desempeño en modo autónomo —sin los atributos jurídicos anotados— porque ese es el escenario real de producción, en el que el quejoso redacta libremente sin conocer la normativa.

Los resultados numéricos de las ocho configuraciones, las matrices de confusión del modelo seleccionado y el análisis de los casos mal clasificados se presentan más adelante en esta misma sección.

Una advertencia metodológica debe acompañar a cualquier lectura de estas métricas. Conforme a lo declarado en la Sección 2.5, el corpus con el que se entrenó y validó el clasificador es un conjunto sintético construido por el equipo de desarrollo, porque a la fecha de elaboración de este documento la Defensoría no ha entregado los expedientes históricos reales y anonimizados solicitados. En consecuencia, los valores reportados caracterizan el comportamiento del modelo sobre la distribución de ese corpus, no sobre la distribución real de las quejas que recibe la institución, y deben considerarse preliminares y sujetos a recalibración.

6.6. Pruebas y verificación

La verificación del sistema se realizó en cuatro niveles, cada uno atendiendo un tipo distinto de defecto.

Verificación de compilación. Cada microservicio se compila y empaqueta antes de desplegarse, y cada aplicación web se compila en modo de producción. Este nivel detecta errores de tipo y de plantilla, que en un lenguaje de tipado estático constituyen una fracción significativa de los defectos posibles.

Verificación de puntos de acceso. Cada punto de acceso se probó de forma individual contra el servicio en ejecución, verificando el código de respuesta y la estructura del cuerpo. Este nivel detecta errores de configuración de seguridad —un punto de acceso que debía ser público y exige token, o al contrario— que la compilación no puede detectar.

Verificación de integración a través del proxy. Cada ruta pública se probó desde el exterior, atravesando la cadena completa: proxy inverso, cortafuegos, microservicio y base de datos. Este nivel es el que detecta los problemas de enrutamiento y de cortafuegos, y fue el que permitió aislar los incidentes descritos en la Sección 6.4. El criterio de verificación aquí es específico: una respuesta de error generada por el servicio de destino constituye una prueba exitosa de conectividad, mientras que una expiración del tiempo de espera indica un problema en el camino.

Verificación funcional de los flujos. Los flujos completos de cada actor se ejecutaron manualmente desde el navegador, siguiendo los casos de uso de la Sección 5.4: presentar una queja sin cuenta, activarla, iniciar sesión, editar, validar desde recepción, rechazar, turnar y consultar el historial exportado.

Un límite de este esquema de verificación debe declararse con claridad: el proyecto no cuenta con pruebas automatizadas —unitarias, de integración o de extremo a extremo— ni con un flujo de integración continua que las ejecute. Toda la verificación descrita se realizó de forma manual. Esto significa que una modificación futura puede reintroducir un defecto ya corregido sin que nada lo detecte de forma automática, y constituye la deuda técnica más relevante del proyecto en materia de calidad.

6.7. Estado de avance por componente

La Tabla 6.4 presenta el balance de lo implementado al cierre de esta etapa. Se incluye de forma deliberada la columna de lo no implementado, porque el valor de este balance para quien retome el proyecto está en lo que falta, no en lo que ya está.

Tabla 6.4: Estado de avance por componente al cierre de la etapa

Componente	Estado	Observaciones
Portal del quejoso	Operativo	Los diecisiete requerimientos funcionales del actor están implementados y desplegados
Panel de revisión	Operativo	Los trece requerimientos del actor de recepción están implementados
Consola de administración	Operativo	Los nueve requerimientos de administración están implementados
Servicios de análisis, subdefensoría y resolución	No iniciado	Los tres roles existen como cuentas y como catálogo de turnado, pero no cuentan con interfaz ni permisos propios

Componente	Estado	Observaciones
Integración del clasificador	Pendiente	El componente se desarrolló y evaluó de forma independiente; su exposición como servicio de la plataforma queda para la etapa siguiente
Cierre de sesión por inactividad	Pendiente	La expiración del token limita la ventana de exposición, pero no equivale a un cierre por inactividad
Pruebas automatizadas	No iniciado	Toda la verificación es manual; no existe integración continua
Corrección de hallazgos de análisis estático	Pendiente	Las vulnerabilidades de neutralización de entradas y de generación pseudoaleatoria identificadas requieren remediación prioritaria
Separación de los cortafuegos por servidor	Pendiente	Ambos servidores comparten el mismo grupo de reglas; no hay exposición real hoy, pero conviene separarlos

7. Módulos de la Plataforma

Este capítulo recorre los módulos entregados desde la perspectiva de quien los usa. Mientras el Capítulo 6 describe cómo se construyó el sistema por capas técnicas, aquí se describe qué hace cada aplicación, pantalla por pantalla, y cómo esas pantallas se corresponden con los casos de uso definidos en la Sección 5.4. El orden sigue el recorrido natural de la información dentro de la institución: entra por el portal del quejoso, se procesa en el panel de revisión y se sostiene desde la consola de administración.

7.1. Portal del quejoso

El portal del quejoso es la única aplicación de acceso público y, por lo tanto, la que define la experiencia que la comunidad politécnica tiene de la Defensoría en línea. Su diseño parte de una premisa que condicionó todo lo demás: la persona que necesita presentar una queja no debería tener que crear una cuenta antes de poder hacerlo.

7.1.1. Área pública

El área pública es accesible sin credenciales y agrupa cuatro funciones.

La **página de inicio** presenta la identidad institucional y tres accesos directos: presentar una queja, consultar el estado de una queja existente e iniciar sesión. Incorpora además una sección de servicios en línea que replica los accesos de referencia del sitio institucional de la Defensoría, de modo que el portal se perciba como continuación del servicio existente y no como un sistema desconectado.

El **formulario de registro de queja** implementa el caso de uso CU-Q01 y es la pantalla más compleja del portal. Está organizada en secciones numeradas: datos del quejoso, datos de la queja, evidencias y, cuando corresponde, datos del tutor. Tres decisiones de diseño merecen mención. La unidad académica donde ocurrieron los hechos se selecciona del catálogo institucional en lugar de capturarse como texto libre, lo que hace el dato comparable entre expedientes. Las evidencias admiten varios archivos con lista previa de lo seleccionado y opción de retirar alguno antes de enviar. Y el aviso normativo sobre el plazo para presentar una queja se presen-

ta en un elemento flotante consultable en cualquier momento, en lugar de un bloque fijo que compitiera por el espacio del formulario.

El **tratamiento de menores de edad** se resuelve dentro del mismo formulario. Cuando la fecha de nacimiento indica que el quejoso es menor, el sistema exige los datos de un tutor o adulto responsable antes de permitir el envío, conforme a la regla RN-Q01. Una vez capturados, el formulario muestra un aviso persistente que confirma que ya se registraron y ofrece editarlos, porque en una versión previa no existía ninguna señal visual de que ese paso se hubiera completado y los usuarios volvían a abrir el diálogo por duda.

La **consulta por folio** implementa el caso de uso CU-Q02 y permite conocer el estado de un expediente con el folio y el correo con que se presentó, sin necesidad de cuenta. Es la funcionalidad que resuelve de forma más directa el problema de trazabilidad que motivó el proyecto.

Completa el área pública un **menú de preguntas frecuentes** sobre la Defensoría y el proceso de queja, cuyo contenido administra el personal de sistemas sin intervención del equipo de desarrollo.

7.1.2. Panel autenticado

Tras activar una cuenta con el folio de una queja previa, el quejoso accede a un panel con seis secciones.

- **Resumen.** Conteo de las quejas propias agrupadas por estatus, como vista de entrada al panel.
- **Mis Quejas.** Listado con filtros por estatus y por texto, y detalle de cada expediente con su línea de tiempo. Desde el detalle se editan los campos habilitados, pero únicamente mientras la queja no haya iniciado su validación: al pasar a validación los campos se bloquean, y el servidor rechaza la modificación aunque se intente por otra vía.
- **Nueva Queja.** Registro de una queja adicional sin volver a capturar la identidad, que se toma de la sesión.
- **Acuerdos de Conciliación.** Consulta de los acuerdos propuestos por la Defensoría, con la posibilidad de aceptarlos o rechazarlos y añadir un comentario.
- **Notificaciones.** Historial persistente de avisos —accesos, quejas creadas, cambios de estatus, acuerdos— con contador de no leídas que se actualiza al marcarlas.
- **Perfil.** Edición de los datos de contacto. El nombre, el correo institucional y la boleta o número de empleado se muestran en modo de solo lectura, porque provienen de la activación de la cuenta y modificarlos rompería la correspondencia con las quejas ya presentadas.

7.2. Panel de revisión

El panel de revisión es la herramienta de trabajo diario del personal de recepción y el punto donde una queja deja de ser un formulario recibido para convertirse en un expediente en trámite. Su diseño se organiza alrededor de una secuencia fija: revisar, decidir, registrar.

7.2.1. Bandeja de entrada

La bandeja es la pantalla de inicio y presenta contadores de quejas pendientes, en proceso y turnadas en el día, junto con la lista de expedientes por trabajar. La elección de encabezar con contadores responde a una necesidad expresada en las sesiones de levantamiento: el personal necesita saber de un vistazo cuánta carga tiene antes de empezar a abrir expedientes.

7.2.2. Detalle y validación

Al abrir un expediente, el personal accede al resumen completo de la queja y a sus documentos adjuntos, que puede previsualizar o descargar. La pantalla incorpora además una **búsqueda de antecedentes**: otras quejas previas de la misma persona. Esta función atiende un problema real del proceso, la duplicidad de expedientes cuando alguien presenta la misma queja por distintos canales, y su ubicación en la pantalla de decisión es deliberada: el dato debe estar disponible en el momento de decidir, no en un módulo separado que haya que consultar por iniciativa propia.

Desde esta pantalla el personal toma una de dos decisiones excluyentes.

El **rechazo** exige seleccionar al menos un motivo y admite observaciones libres. Al confirmarlo, el expediente pasa a estado rechazado y el sistema envía de forma automática el correo correspondiente al quejoso. La regla RN-R03 hace que el rechazo nunca sea una acción silenciosa: no es posible descartar un expediente sin que su autor se entere ni sin dejar registrado el motivo.

El **turnado** exige elegir el área o dependencia —tomada del catálogo institucional— y el defensor o subdefensor responsable, tomado del catálogo de personal activo con esos roles. Admite comentarios y genera el folio oficial de turnado, distinto del folio de seguimiento que conoce el quejoso.

En ambos casos el sistema registra quién procesó el expediente y cuándo, sin excepción, conforme al requerimiento RNF-R01.

7.2.3. Registro manual de quejas en papel

Una parte del volumen que recibe la Defensoría llega en papel. El módulo de registro manual permite incorporar esos expedientes al mismo flujo digital, capturando la identidad del quejoso, la dependencia, el número de oficio, la fecha de recepción física, el tipo de documento, la descripción, la ubicación física del expediente y, de forma opcional, el archivo digitalizado.

Este módulo tiene una particularidad de diseño: admite que no se conozca la boleta o el número de empleado del quejoso, situación que el registro propio no permite. La razón es que un documento en papel puede provenir de una persona externa o llegar sin esos datos, y rechazarlo por ello significaría dejar fuera del sistema quejas que la institución sí recibió. El expediente queda marcado con origen manual, de modo que su procedencia siempre es identificable.

7.2.4. Historial y exportación

El historial reúne los expedientes ya procesados —turnados o rechazados— con filtros por texto, estatus y fecha. Su función principal no es la consulta puntual, que ya resuelve la búsqueda de antecedentes, sino la generación de reportes: el resultado filtrado se exporta a un archivo de hoja de cálculo que se abre directamente en las herramientas ofimáticas que el personal ya usa, sin conversión previa.

7.2.5. Emisión de acuerdos de conciliación

El personal puede emitir un acuerdo de conciliación dirigido al quejoso de un expediente, capturando asunto y términos. El acuerdo queda en estado pendiente y aparece de inmediato en el panel del quejoso, que puede aceptarlo o rechazarlo. Este módulo es el punto donde el flujo de la plataforma se conecta con las etapas del procedimiento que aún no tienen interfaz propia.

7.3. Consola de administración

La consola de administración es la que hace que la plataforma sea operable a largo plazo sin intervención del equipo de desarrollo. Agrupa cuatro frentes independientes entre sí.

7.3.1. Usuarios y roles

Ciclo de vida completo de las cuentas del personal: alta con contraseña temporal generada por el sistema, edición, restablecimiento de contraseña, baja y reactivación, para cualquiera de los cinco roles.

Dos decisiones de diseño de este módulo tienen consecuencias de seguridad. La primera es que las cuentas nuevas se crean siempre como temporales y con obligación de cambiar la contraseña en el primer acceso, de modo que la contraseña generada por el sistema nunca sea la contraseña permanente de nadie. La segunda es que dar de baja una cuenta la desactiva en lugar de eliminarla: si se eliminara, las acciones que esa cuenta registró en la bitácora quedarían sin autor identificable, lo que anularía el propósito de la auditoría.

7.3.2. Catálogo de unidades académicas

Administración del catálogo institucional que consumen el formulario público de quejas y el turnado del panel de revisión. Permite el alta y la edición individual, y la importación masiva desde un archivo de hoja de cálculo con el formato institucional.

La importación masiva se diseñó para ser tolerante a errores: en lugar de rechazar el archivo completo cuando una fila está mal formada, procesa lo que puede y devuelve un resumen de filas procesadas, creadas, actualizadas y con error. El criterio detrás de esa decisión es que un archivo de doscientos registros con un error tipográfico en uno de ellos no debería obligar a repetir toda la carga.

7.3.3. Plantillas de documentos oficiales

Administración del contenido reutilizable de los oficios que genera el procedimiento. Cada plantilla admite marcadores que el sistema sustituye por los datos del expediente al generar el documento, y la consola permite consultar qué marcadores están disponibles, previsualizar el resultado con datos de ejemplo y publicar los cambios.

Cada plantilla se identifica por una clave estable de negocio y no por su identificador autogenerado, de modo que el resto del sistema pueda referenciarla sin depender del orden en que se hayan creado. Cada actualización registra quién la hizo y cuándo.

7.3.4. Seguridad y respaldos

Este módulo agrupa la operación más sensible de la plataforma. Permite listar los respaldos existentes, ejecutar un respaldo bajo demanda, descargar uno específico y restaurar la base de datos a partir de él.

La restauración es la única operación destructiva de todo el sistema y recibe un tratamiento diferenciado: exige una confirmación explícita en el cuerpo de la petición, de modo que no pueda dispararse por accidente con una sola acción de la interfaz. Además, la capacidad de generar, descargar y restaurar respaldos está reservada al rol de administración de sistemas, que es el único que dispone de esos puntos de acceso.

El módulo incluye también la **bitácora de acciones críticas**, que registra las altas, bajas, ediciones y reactivaciones de cuentas, los restablecimientos de contraseña, las actualizaciones de plantillas, los respaldos manuales, las restauraciones y los inicios de sesión, con el usuario que los ejecutó, la dirección de origen y la fecha. Esta bitácora es el mecanismo que hace verificable el requerimiento RNF-A02 y, junto con la regla de que las cuentas se desactivan en lugar de eliminarse, constituye el soporte de auditoría de la plataforma.

7.3.5. Administración de preguntas frecuentes

Alta, edición y eliminación del contenido del menú de preguntas del portal público. Los cambios se reflejan de inmediato en el portal, sin necesidad de recompilar ni redespargar la aplicación del quejoso, lo que traslada el mantenimiento de ese contenido del equipo de desarrollo al personal de la institución.

7.4. Relación entre módulos y casos de uso

La Tabla 7.1 presenta la correspondencia entre los módulos entregados y los casos de uso especificados en la Sección 5.4. Su función es de trazabilidad: permite verificar que cada caso de uso especificado tiene un módulo que lo realiza, y que ningún módulo se construyó sin un caso de uso que lo justifique.

Tabla 7.1: Correspondencia entre módulos entregados y casos de uso

Módulo	Casos de uso	Requerimientos
Registro público de quejas	CU-Q01	RF-Q02, RF-Q03, RF-Q15
Consulta por folio	CU-Q02	RF-Q01
Activación de cuenta y acceso	CU-Q03, CU-Q04, CU-Q05	RF-Q04, RF-Q05, RF-Q06
Panel del quejoso	CU-Q06, CU-Q07, CU-Q09, CU-Q10	RF-Q07 a RF-Q11, RF-Q13, RF-Q14
Acuerdos de conciliación (quejoso)	CU-Q08	RF-Q12
Preguntas frecuentes	—	RF-Q16
Bandeja de entrada	CU-R02	RF-R02
Detalle, rechazo y turnado	CU-R03, CU-R04	RF-R03 a RF-R07, RF-R12
Registro manual	CU-R05	RF-R08
Historial y exportación	CU-R06	RF-R09, RF-R10

Módulo	Casos de uso	Requerimientos
Emisión de acuerdos	CU-R07	RF-R11
Usuarios y roles	CU-A02, CU-A03, CU-A04	RF-A03
Catálogo institucional	CU-A05	RF-A04
Plantillas de documentos	CU-A06	RF-A05
Seguridad y respaldos	CU-A07, CU-A08, CU-A09	RF-A06, RF-A07
Administración de preguntas	CU-A10	RF-A08

Referencias

- [1] Instituto Politécnico Nacional. «Ley Orgánica del Instituto Politécnico Nacional,» dirección: <https://www.ipn.mx/assets/files/defensoria/docs/manuales/ley-org%C3%A1nica-ipn.pdf> (visitado 10-04-2026).
- [2] Instituto Politécnico Nacional, «Acuerdo de creación de la Defensoría de los Derechos Politécnicos,» *Gaceta Politécnica*, n.º 622 (Extraordinaria), págs. 3-7, ene. de 2006. dirección: <https://www.ipn.mx/assets/files/imageninstitucional/docs/gaceta-extraordinaria/2006/01/622-31-enero-gaceta-extra-.pdf> (visitado 10-04-2026).
- [3] Instituto Politécnico Nacional, «Manual de Organización de la Defensoría de los Derechos Politécnicos,» Dirección de Planeación y Organización, Ciudad de México, México, oct. de 2022. dirección: <https://www.ipn.mx/assets/files/defensoria/docs/manuales/moddp2022.pdf> (visitado 10-04-2026).
- [4] Instituto Politécnico Nacional, «Manual de Procedimientos de la Defensoría de los Derechos Politécnicos,» Dirección de Planeación y Organización, Ciudad de México, México, oct. de 2024. dirección: <https://www.ipn.mx/assets/files/defensoria/docs/manuales/mp-ddp.pdf> (visitado 10-04-2026).
- [5] Secretaría de la Función Pública. «Sistema Integral de Denuncias Ciudadanas (SIDECA),» Gobierno de México, dirección: <https://sideca.buengobierno.gob.mx/> (visitado 10-04-2026).
- [6] Gobierno de la Ciudad de México. «Denuncia Digital,» Fiscalía General de Justicia de la Ciudad de México, dirección: <https://denunciadigital.cdmx.gob.mx/> (visitado 10-04-2026).
- [7] Whistlelink. «The All-in-One Whistleblowing Solution.» (2026), dirección: <https://www.whistlelink.com/> (visitado 10-04-2026).
- [8] EQS Group. «EQS Integrity Line: The leading whistleblowing system in Europe.» (2026), dirección: <https://www.eqs.com/es/soluciones-gobierno-corporativo/integrity-line/> (visitado 10-04-2026).
- [9] NAVEX Global. «WhistleB: Whistleblowing Solutions for a Modern Workplace.» (2026), dirección: <https://www.navex.com/en-us/products/whistleb/> (visitado 10-04-2026).
- [10] Universidad Nacional Autónoma de México. «Sistema de Denuncia en Línea de la Defensoría de los Derechos Universitarios,» dirección: <https://www.defensoria.unam.mx/denuncia/> (visitado 10-04-2026).
- [11] C. Cortes y V. Vapnik, «Support-Vector Networks,» *Machine Learning*, vol. 20, n.º 3, págs. 273-297, sep. de 1995. DOI: [10.1007/BF00994018](https://doi.org/10.1007/BF00994018).
- [12] L. Breiman, «Random Forests,» *Machine Learning*, vol. 45, n.º 1, págs. 5-32, oct. de 2001. DOI: [10.1023/A:1010933404324](https://doi.org/10.1023/A:1010933404324).
- [13] N. Reimers e I. Gurevych, «Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks,» en *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing (EMNLP-IJCNLP)*, Hong Kong, China, nov. de 2019, págs. 3982-3992. dirección: <https://arxiv.org/abs/1908.10084>.
- [14] Amazon Web Services. «¿Qué son los microservicios?» Dirección: <https://aws.amazon.com/es/microservices/> (visitado 10-04-2026).

- [15] M. Villamizar et al., «Evaluating the Monolithic and the Microservice Architecture Pattern to Deploy Web Applications in the Cloud,» Universidad de los Andes, Colombia.
- [16] Agencia Mexicana de Cooperación Internacional para el Desarrollo. «Catálogo de Capacidades Mexicanas: Sistema Integral de Quejas y Denuncias Ciudadanas (SIDECA),» Secretaría de Relaciones Exteriores, dirección: <https://de.sre.gob.mx/capacidades/instituciones-mexicanas/sfp/sistema-integral-de-quejas-y-denuncias-ciudadanas-sidec> (visitado 10-04-2026).
- [17] Secretaría Anticorrupción y Buen Gobierno. «Implementan SFP, BID y GovLab proyecto de mejora al Sistema Integral de Denuncias Ciudadanas (SIDECA),» Gob.mx. (jul. de 2018), dirección: <https://www.gob.mx/buengobierno/prensa/implementan-sfp-bid-y-govlab-proyecto-de-mejora-al-sistema-integral-de-denuncias-ciudadanas-sidec> (visitado 10-04-2026).
- [18] Denuncia.org. «Denuncia en línea: Consulta si en tu estado la Fiscalía cuenta con la opción de denuncia digital,» Impunidad Cero. (2026), dirección: <https://denuncia.org/denuncia-digital/> (visitado 10-04-2026).
- [19] Fiscalía General de Justicia de la Ciudad de México. «Servicios en Línea: Denuncia Digital, Denuncia Anónima y MP Transparente.» (2026), dirección: <https://www.fgjcdmx.gob.mx/servicios/en-linea> (visitado 10-04-2026).
- [20] Gobierno de la Ciudad de México. «MP Virtual: Ratificación de Querellas y Actas Especiales,» Fiscalía General de Justicia de la Ciudad de México, dirección: <https://mpvirtual.fgjcdmx.gob.mx/CiberDenuncia/TerminosCondiciones.aspx> (visitado 10-04-2026).
- [21] Nación321. «5 apps que nos ayudan a combatir la corrupción,» Grupo Multimedia Lauman, SAPI de CV. (feb. de 2024), dirección: <https://www.nacion321.com/ciudadanos/5-apps-que-nos-ayudan-a-combatir-la-corrupcion> (visitado 10-04-2026).
- [22] Secretaría de la Contraloría del Gobierno del Estado de México. «Sistema de Atención Mexiquense (SAM).» (2026), dirección: <https://www.secogem.gob.mx/sam/tramite.asp> (visitado 10-04-2026).
- [23] Whistleblowing Solutions ApS. «The simple, secure, and compliant whistleblowing system,» Whistleblower Software. (2026), dirección: <https://whistleblowersoftware.com/es> (visitado 10-04-2026).
- [24] Bully Button. «The Bully Button: Reporting and Prevention System.» (2026), dirección: <http://bullybutton.com/> (visitado 10-04-2026).
- [25] Secretaría de Educación Pública. «Ventanilla Escolar: Orientación y Quejas,» Gobierno de México, dirección: <https://www.gob.mx/sep/acciones-y-programas/ventanilla-escolar> (visitado 10-04-2026).
- [26] Instituto Politécnico Nacional. «Protocolo Alerta IPN: Seguridad Institucional,» Secretaría General del IPN, dirección: <https://www.ipn.mx/seguridad/> (visitado 10-04-2026).
- [27] BBVA. «Arquitectura de software o sistemas: ¿qué es y ejemplos?» (2021), dirección: <https://www.bbva.com/es/innovacion/arquitectura-de-software-o-sistemas-que-es-y-ejemplos/> (visitado 10-04-2026).
- [28] UTEL. «Atributos de calidad del software,» SCALA, dirección: https://gc.scalahed.com/recursos/files/r161r/w26023w/L1IDS118_l2_s3.pdf (visitado 10-04-2026).

- [29] Microsoft. «¿Qué es la escalabilidad?» Microsoft Learn. (2023), dirección: <https://learn.microsoft.com/es-es/biztalk/core/what-is-scalability> (visitado 10-04-2026).
- [30] Amazon Web Services. «Construcción de una arquitectura distribuida y escalable,» dirección: <https://aws.amazon.com/startups/learn/building-scalable-distributed-architecture?lang=es> (visitado 10-04-2026).
- [31] M. T. Nygard, *Release It!: Design and Deploy Production-Ready Software*. Pragmatic Bookshelf.
- [32] IBM. «¿Qué es una arquitectura monolítica?» Dirección: <https://www.ibm.com/think/topics/monolithic-architecture> (visitado 10-04-2026).
- [33] A. S. Tanenbaum et al., *Arquitectura de Sistemas Distribuidos*.
- [34] IBM. «¿Qué es la arquitectura de tres capas?» Dirección: <https://www.ibm.com/mx-es/think/topics/three-tier-architecture> (visitado 10-04-2026).
- [35] Microsoft Ibérica, *Guía de Arquitectura N-Capas orientada al Dominio con .NET 4.0*.
- [36] B. M. Coti Colop, «Reglas del Negocio en Arquitectura Tres Capas,» Tesis de mtría., Universidad de San Carlos de Guatemala.
- [37] J. L. Poza Luján, «Propuesta de arquitectura distribuida de control inteligente basada en políticas de calidad de servicio,» Tesis doct., Universitat Politècnica de València.
- [38] R. T. Fielding, «Architectural Styles and the Design of Network-based Software Architectures,» Tesis doct., University of California, Irvine.
- [39] B. Stopford, *Designing Event-Driven Systems: Concepts and Patterns for Streaming Services with Apache Kafka*. O'Reilly Media.
- [40] Spring Cloud. «Spring Cloud Config & Netflix Eureka,» VMware, Inc., dirección: <https://spring.io/projects/spring-cloud> (visitado 10-04-2026).
- [41] IBM. «¿Qué es PostgreSQL?» Dirección: <https://www.ibm.com/mx-es/think/topics/postgresql> (visitado 10-04-2026).
- [42] H. J. Schönig, *Mastering PostgreSQL 15: Advanced techniques to build and manage scalable, reliable, and fault-tolerant database applications*. Packt Publishing.
- [43] M. Kleppmann, *Designing Data-Intensive Applications: The Big Ideas Behind Reliable, Scalable, and Maintainable Systems*. O'Reilly Media.
- [44] Oracle. «¿Qué es la tecnología Java y para qué la necesito?» Dirección: https://www.java.com/es/download/help/whatis_java.html (visitado 10-04-2026).
- [45] IBM. «¿Qué es Java?» Dirección: <https://www.ibm.com/mx-es/think/topics/java> (visitado 10-04-2026).
- [46] IBM. «¿Qué es Spring Boot?» Dirección: <https://www.ibm.com/mx-es/think/topics/java-spring-boot> (visitado 10-04-2026).
- [47] C. Álvarez Caules. «Spring Boot Actuator,» Arquitectura Java, dirección: <https://www.arquitecturajava.com/spring-boot-actuator/> (visitado 10-04-2026).
- [48] Amazon Web Services. «¿Qué es Python?» Dirección: <https://aws.amazon.com/es/what-is/python/> (visitado 10-04-2026).
- [49] Angular. «What is Angular?» Dirección: <https://angular.dev/overview> (visitado 10-04-2026).

- [50] Imagina Formación. «Conoce las ventajas de utilizar Angular,» dirección: <https://imaginaformacion.com/tutoriales/conoce-las-ventajas-de-utilizar-angular> (visitado 10-04-2026).
- [51] Red Hat. «¿Qué es Docker?» Dirección: <https://www.redhat.com/es/topics/containers/what-is-docker> (visitado 10-04-2026).
- [52] B. Burns, J. Beda y K. Hightower, *Kubernetes: Up and Running: Dive into the Future of Infrastructure*. O'Reilly Media.
- [53] NGINX. «nginx,» dirección: <https://nginx.org/en/> (visitado 10-04-2026).
- [54] B. Khanal. «Nginx: The Lightweight Powerhouse for Modern Web Serving,» Medium. (2023), dirección: <https://medium.com/@bishal.khanal.2057/nginx-the-lightweight-powerhouse-for-modern-web-serving-717f7cc628da> (visitado 10-04-2026).
- [55] GitHub. «Acerca de GitHub y Git,» dirección: <https://docs.github.com/es/get-started/start-your-journey/about-github-and-git> (visitado 10-04-2026).
- [56] E. Linis. «Conceptos Básicos de Git,» GitHub Gist, dirección: <https://gist.github.com/erlinis/57a55dfb0337f5cd15cd> (visitado 10-04-2026).
- [57] Spring Security. «OAuth 2.0 Resource Server JWT,» VMware, Inc., dirección: <https://docs.spring.io/spring-security/reference/servlet/oauth2/resource-server/jwt.html> (visitado 10-04-2026).
- [58] E. Rescorla, «The Transport Layer Security (TLS) Protocol Version 1.3,» Internet Engineering Task Force (IETF), RFC 8446.
- [59] C. D. Manning, P. Raghavan y H. Schütze, *Introduction to Information Retrieval*. Cambridge, Reino Unido: Cambridge University Press, 2008. dirección: <https://nlp.stanford.edu/IR-book/>.
- [60] D. Jurafsky y J. H. Martin, *Speech and Language Processing*, 3.^a ed. Stanford University, 2023, Borrador. dirección: <https://web.stanford.edu/~jurafsky/slp3/>.
- [61] T. Mikolov, I. Sutskever, K. Chen, G. S. Corrado y J. Dean, «Distributed Representations of Words and Phrases and their Compositionality,» en *Advances in Neural Information Processing Systems 26 (NIPS 2013)*, 2013, págs. 3111-3119. dirección: <https://arxiv.org/abs/1310.4546>.
- [62] M. Honnibal, I. Montani, S. V. Landeghem y A. Boyd, *spaCy: Industrial-strength Natural Language Processing in Python*, 2020. DOI: [10.5281/zenodo.1212303](https://doi.org/10.5281/zenodo.1212303).
- [63] A. Vaswani, N. Shazeer, N. Parmar et al., «Attention is All You Need,» en *Advances in Neural Information Processing Systems 30 (NIPS 2017)*, 2017, págs. 5998-6008. dirección: <https://arxiv.org/abs/1706.03762>.
- [64] HuggingFace. «paraphrase-multilingual-mpnet-base-v2,» HuggingFace Model Hub. (2023), dirección: <https://huggingface.co/sentence-transformers/paraphrase-multilingual-mpnet-base-v2> (visitado 10-04-2026).
- [65] T. G. Dietterich, «Ensemble Methods in Machine Learning,» en *Multiple Classifier Systems. MCS 2000. Lecture Notes in Computer Science*, vol. 1857, Berlín, Alemania: Springer, 2000, págs. 1-15. DOI: [10.1007/3-540-45014-9_1](https://doi.org/10.1007/3-540-45014-9_1).
- [66] F. Pedregosa et al., «Scikit-learn: Machine Learning in Python,» *Journal of Machine Learning Research*, vol. 12, págs. 2825-2830, 2011. dirección: <http://www.jmlr.org/papers/volume12/pedregosa11a/pedregosa11a.pdf>.